

Design and Analysis of Software Systems (DASS)

Complete Exam Study Guide

Spring 2026 — All Lectures Covered

Topics Covered:

- Introduction to Software Engg.
- Process Models
- Project Estimation
- Project Scheduling & Tracking
- Conceptual Modeling
- Introduction to Design
- UI/UX & Usability Design
- GRASP & Design Quality
- Refactoring
- Legacy Systems & Re-engineering
- SDLC
- Agile & Scrum
- Requirements Engineering
- Use Case Modeling & UML
- Software Testing
- Cybersecurity & Secure Design
- Behavioral Modeling (Seq/State)
- Case Studies & Code Quality
- Software Maintenance

IMPORTANT: This guide covers ALL lectures from 5-Jan through 16-April. Study every section carefully!

Contents

1	Introduction to Software Engineering	5
1.1	Why Software Engineering Matters	5
1.2	Green Field vs. Brown Field Systems	5
1.3	Five Pillars of Software Engineering (from Class)	5
1.4	Systems Thinking	6
2	Software Development Life Cycle (SDLC)	7
2.1	Phases of the SDLC	7
2.2	Detailed Phase Breakdown	8
2.3	The PDCA Cycle (Plan–Do–Check–Act)	8
2.4	Typical Time Distribution	8
3	Software Process Models	10
3.1	Waterfall Model	10
3.2	Prototyping Model	10
3.3	Evolutionary / Incremental Model	10
3.4	Spiral Model	11
3.5	Comparison of Process Models	11
3.6	The Cone of Uncertainty	11
4	Design Principles	12
4.1	SOLID Principles	12
4.2	Open-Closed Principle (OCP) — In Detail	12
4.3	GRASP Principles	13
4.4	Coupling & Cohesion	13
4.5	Types of Software Maintenance	13
5	Agile Development	15
5.1	The Agile Manifesto (2001)	15
5.2	12 Agile Principles (Key Ones)	15
5.3	Scrum Framework	15
5.4	User Stories	16
5.5	Key Agile Practices	16
5.6	DevOps	17
5.7	Agile Process Spectrum	17
6	Project Estimation & Scheduling	18
6.1	The Iron Triangle of Project Management	18
6.2	Function Point Analysis (FPA)	18
6.3	Value Adjustment Factor (VAF)	19
6.4	Work Breakdown Structure (WBS)	19
6.5	Gantt Charts & Critical Path	20
6.6	Earned Value Management (EVM)	21
7	Requirements Engineering	22
7.1	Functional vs. Non-Functional Requirements	22
7.2	Requirements Gathering Techniques	22
7.3	Qualities of Good Requirements	22
7.4	Categorizing Requirements: Basic, Differentiators, Delights	23

7.5	MoSCoW Prioritization	23
7.6	Minimum Viable Product (MVP)	23
7.7	Verification vs. Validation	23
7.8	Software Requirements Specification (SRS)	24
7.9	Data Privacy & Regulations	24
8	Use Case Modeling	25
8.1	Actors	25
8.2	Use Case Template	25
8.3	Use Case Relationships	26
8.4	Granularity of Use Cases	26
9	Conceptual Modeling & UML	27
9.1	From Use Cases to Conceptual Model	27
9.2	UML Class Diagram Notation	27
9.3	Access Modifiers	27
9.4	Associations & Multiplicity	28
9.5	Types of Attributes	28
9.6	Key Distinction: Conceptual vs. Software Classes	28
10	Software Testing	29
10.1	Levels of Testing	29
10.2	Integration Testing Strategies	30
10.3	Black Box vs. White Box Testing	30
10.4	Black Box Testing Techniques	30
10.4.1	Equivalence Partitioning	30
10.4.2	Boundary Value Analysis	30
10.4.3	State Transition Testing	31
10.5	White Box Testing Techniques	31
10.6	Test Case	31
10.7	Types of Acceptance Testing	31
10.8	Key Testing Principles	32
10.9	Testing Metrics	32
10.10	Qualities of a Good Tester	32
10.11	Regression Testing & Maintenance Testing	32
11	Introduction to Design	33
11.1	Design vs. Coding: A Critical Distinction	33
11.2	Models and Modeling Languages	33
11.3	Core Design Principles	34
11.3.1	Abstraction	34
11.3.2	Separation of Concerns	34
11.3.3	Modularity	34
11.3.4	Goal: Loose Coupling + High Cohesion	34
11.4	Design Smells	34
11.5	Practice & Review — Introduction to Design	35
12	Cybersecurity and Secure Design	36
12.1	Non-Functional Requirements and Security	36
12.2	Zero Trust Security Model	36
12.3	Cybersecurity Risk Management	37
12.4	Security Testing	37

12.5 Case Study — World Bank: Cash Transaction Processing	38
12.6 Case Study — Pharmacy Insurance Claims System	38
12.7 Database Normalization and Data Integrity	39
12.8 Network Security Fundamentals	39
12.9 Open-Source Software Security	39
12.10 Secure Coding Practices	40
12.11 Digital Public Infrastructure	40
12.12 Practice & Review — Cybersecurity and Secure Design	41
13 UI/UX Design and Usability	42
13.1 UI vs. UX: A Clear Distinction	42
13.2 Trade-offs in Design	42
13.3 Designing for the User	42
13.3.1 Personas	42
13.3.2 Mental Models & Intuition	43
13.3.3 Cognitive Load	43
13.4 Information Architecture	43
13.5 Outside-In Design	43
13.6 Patterns and Heuristics	43
13.7 Practice & Review — UI/UX and Usability	43
14 Behavioral Modeling: Sequence Diagrams & State Machines	45
14.1 Sequence Diagrams	45
14.1.1 Purpose and Elements	45
14.1.2 System Sequence Diagram (SSD) vs. Sequence Diagram	45
14.1.3 Synchronous vs. Asynchronous Messages	45
14.2 State Machines	45
14.3 Case Study: Submarine Chamber (March 26)	46
14.4 Failure Mode & Effects Analysis (FMEA)	46
14.5 Formal Verification & Model Checking	46
14.6 Model-Based Testing	46
14.7 Practice & Review — Behavioral Modeling	47
15 GRASP Patterns & Design Quality	48
15.1 Quality Attributes	48
15.2 Accuracy vs. Correctness	48
15.3 The Nine GRASP Patterns	49
15.4 Coupling Metrics: Fan-In and Fan-Out	49
15.5 Model-View-Controller (MVC)	49
15.6 Practice & Review — GRASP & Design Quality	49
16 Case Studies: When Code Quality Costs Lives	51
16.1 Critical vs. Safety-Critical Systems	51
16.2 Therac-25 (1985–1987)	51
16.3 Ariane 5, Flight 501 (1996)	51
16.4 Boeing 737 MAX MCAS (2018–2019)	51
16.5 Brake Calibration Incident (Lecturer’s Car)	51
16.6 Takeaways for the Software Engineer	52
16.7 AI-Assisted Development: Benefits and Risks	52
16.8 Practice & Review — Case Studies	52

17 Refactoring	54
17.1 Lehman’s Laws of Software Evolution	54
17.2 Why Refactor?	54
17.3 Code Smells Driving Refactoring	54
17.4 Core Refactoring Techniques	55
17.5 Principles of Safe Refactoring	55
17.6 Tools	55
17.7 Design Refactoring	55
17.8 Practice & Review — Refactoring	56
18 Software Maintenance & Legacy Systems	57
18.1 The Four Types of Maintenance	57
18.2 Software Aging	57
18.3 Legacy Systems	57
18.4 Modernising Legacy Systems	58
18.4.1 The Evaluation Quadrant	58
18.5 Reverse Engineering & Forward Engineering	58
18.6 Feathers’ 5-Step Process for Changing Legacy Code	58
18.7 Maintenance Metrics	59
18.8 Layered Architecture for Maintainability	59
18.9 Practice & Review — Maintenance & Legacy	59
19 Exam Traps & Warnings (From Quiz 1 Answer Key)	61
20 Crossword Vocabulary — Key Terms & Definitions	62
20.1 Terms from Quiz 1 Crossword (All Sets)	62
20.2 Additional Likely Crossword Terms (New Topics Since Quiz 1)	62
21 Scenario-Based Practice Questions (Quiz 1 Style)	64
21.1 Practice Scenario 1: Hospital Management System	64
21.2 Practice Scenario 2: E-Commerce Platform	65
21.3 Practice: Use Case & Conceptual Modeling Question	65
21.4 Practice: Testing Question	66
22 Additional Practice Questions	68
22.1 Short Answer Questions	68
22.2 Comparison Questions	70
22.3 Multiple Choice Quick Review	72
23 Quick Reference Cheat Sheet	74

1 Introduction to Software Engineering

What is Software Engineering?

Software Engineering is the application of **engineering principles and techniques** to the **design, development, testing, and maintenance** of software systems. It is NOT just about writing code — it is about building **systems** that are reliable, maintainable, and scalable.

1.1 Why Software Engineering Matters

Key Insight from Class

A simple modern-day game like Candy Crush has **millions of lines of code**. The new Mahindra XEV 9e has **75 million lines of code!** Even at peak productivity (1 line/minute), one engineer can produce only **120,000 lines/year**. Building software is fundamentally about **teamwork**, not individual brilliance.

- **Industry Average:** If you can move **25 lines of code per day** to production, you are a brilliant programmer. 99% of what you write will NOT move to production directly.
- **People First, Not Technology First:** It is about what your code can do for others, and how you can sustain what you write by keeping it as simple as possible.
- **Maintenance dominates:** The development cycle takes $\approx$ **20%** of effort; maintenance takes $\approx$ **80%** of effort.
- Only **10–20%** of software projects are truly successful (on time, within budget, meeting specs).

1.2 Green Field vs. Brown Field Systems

Green Field vs. Brown Field

Type	Description
Green Field	A brand new software system developed from scratch. Very rare in the real world.
Brown Field	An existing software system that is modified, extended, or enhanced. This is what you will encounter 99% of the time in industry.

Remember!

You will almost NEVER start from scratch in the real world. You will always be building on top of existing systems. Understanding existing codebases and being able to extend them is a **critical skill**.

1.3 Five Pillars of Software Engineering (from Class)

The professor emphasized **five key skills** that every software engineer must have:

#	Skill
1	Design — Thinking before coding; systematic approach
2	Teamwork — Complementary skills; collaboration
3	Communication — With team, clients, stakeholders
4	Programming — Writing clean, maintainable code
5	Documentation — So others can understand and maintain your work

1.4 Systems Thinking

Beyond Software Engineering

Modern software is part of **larger systems**. The “S” in DASS stands for **Systems**. You must become a **systems engineer** — someone who understands the entire system (software + hardware + people + processes), not just the code.

Knight Capital Group Case Study (2012)

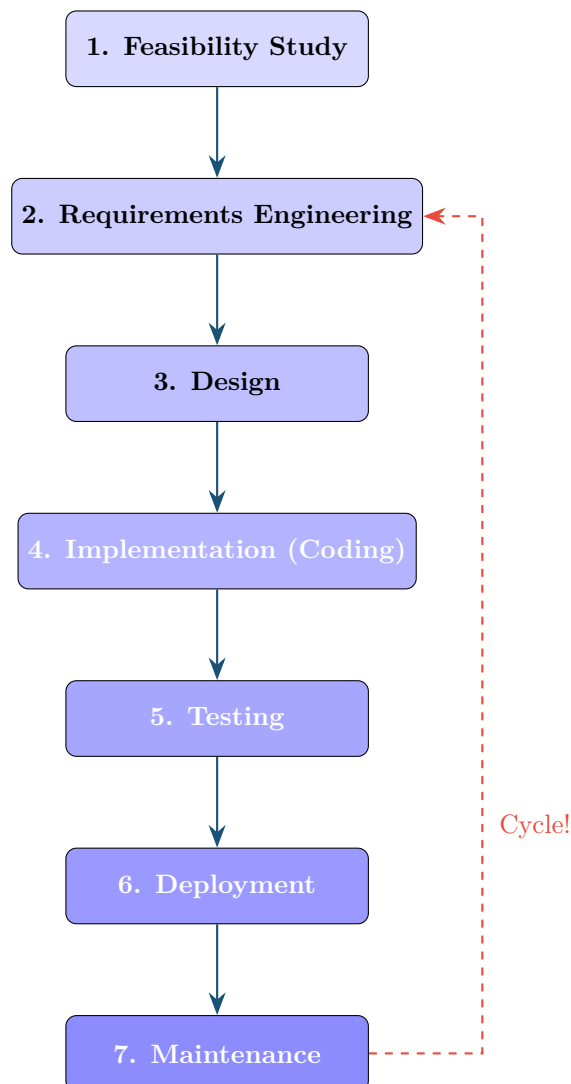
A small software update to a trading system caused a **45-minute outage** that resulted in **millions of dollars in losses**. The company had to be sold because they could not sustain the losses. **Lesson:** Proper process **MUST** be followed during updates and deployments.

2 Software Development Life Cycle (SDLC)

What is SDLC?

The **Software Development Life Cycle (SDLC)** is a systematic framework used to **plan, design, develop, test, and deliver** software applications. It divides the development process into multiple **phases**, each with defined activities and deliverables.

2.1 Phases of the SDLC



2.2 Detailed Phase Breakdown

Phase	Description & Key Activities
Feasibility Study	Evaluate whether the project is viable. Assess technical , economic , operational , and scheduling feasibility. Determine resources, risks, and potential benefits.
Requirements	Gather, analyze, and document what the software must do. Produce the Software Requirements Specification (SRS) document.
Design	Create the blueprint: architecture, components, interfaces. Translate requirements (problem space) into a solution (solution space).
Implementation	Write the actual code. Follow coding standards. Build what was designed.
Testing	Verify that the software meets requirements. Static analysis (linting) + Dynamic analysis (running tests).
Deployment	Release the software to the production environment. Consider the deployment environment carefully.
Maintenance	Update, modify, fix, and enhance the software. This is where 70–80% of the effort and money goes!

Why is it called a “Cycle”?

Because after Version 1.0, the client will want Version 2.0! You go back through the phases: new requirements → updated design → new code → more testing → new release. Software is **never constant** — it keeps evolving. Sub-cycles can occur too (e.g., going back from testing to design if issues are found).

2.3 The PDCA Cycle (Plan–Do–Check–Act)

PDCA / Deming Cycle

A framework for **continuous improvement**:

1. **Plan:** Identify goals and objectives; plan the approach.
2. **Do:** Execute the plan; implement the solution.
3. **Check:** Verify the results; compare with requirements.
4. **Act:** Implement improvements based on what was learned.

This cycle repeats continuously to drive ongoing improvement in the development process.

2.4 Typical Time Distribution

Activity	Approximate Time
Requirements & Design	20–30%
Implementation (Coding)	15–20%
Testing	30–40%
Maintenance	60–80% of total lifetime

Key Takeaway

Coding is only a small part of software engineering. **Understanding code, designing systems, and maintaining them** is far more important in the real world than just writing code fast.

3 Software Process Models

What is a Process Model?

A **process model** defines the **order and manner** in which SDLC phases are carried out. Different models suit different project types, risk levels, and requirements stability.

3.1 Waterfall Model

Waterfall Model (Traditional / Plan-Driven)

How it works: A **linear, sequential** approach. Each phase must be **completed fully** before moving to the next. No going back.

Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance

When to use:

- Requirements are **well-understood and stable**
- The project is **small to medium** in size
- Technology is **well-known**

Strengths: Simple, easy to manage, clear milestones.

Weaknesses: Inflexible; no working software until late; risky if requirements change.

3.2 Prototyping Model

Prototyping Model

How it works: Build a quick **prototype** (mock-up) to test requirements and get user feedback **early**. Refine based on feedback.

When to use:

- Requirements are **unclear or evolving**
- Need to **validate feasibility** early

Strengths: Early user feedback, reduces misunderstanding.

Weaknesses: Prototype may be mistaken for the final product; can lead to scope creep.

3.3 Evolutionary / Incremental Model

Incremental Model

How it works: Break the system into **smaller pieces (increments)**. Develop and deliver each increment one at a time. Each increment adds functionality.

When to use:

- When you want to deliver **partial functionality early**
- Requirements may **evolve** over time

Strengths: Partial working software delivered early; easier to manage.
Weaknesses: Requires good planning to define increments.

3.4 Spiral Model

Spiral Model

How it works: A **risk-driven meta-model** that combines elements of waterfall and prototyping. Each **spiral** (iteration) goes through: Planning → Risk Analysis → Engineering → Evaluation.

When to use:

- Large, **high-risk** projects
- Requirements are complex and may change

Strengths: Explicit risk management; flexible.

Weaknesses: Complex, expensive; requires risk-assessment expertise.

3.5 Comparison of Process Models

Model	Requirements	Risk Level	Flexibility
Waterfall	Stable, well-known	Low	Low
Prototyping	Unclear, evolving	Medium	Medium
Incremental	Partially known	Medium	High
Spiral	Complex, changing	High	Very High
Agile (Scrum)	Evolving, user-driven	Medium–High	Very High

3.6 The Cone of Uncertainty

Cone of Uncertainty

At the **start** of a project, estimates can be off by a factor of **4×** (over or under). As the project progresses and more information is known, estimates and actuals **converge**. This is why shorter delivery cycles and iterative development help — they reduce uncertainty faster.

Technical Debt

Technical debt is the cost of implementing **quick fixes or workarounds** that will need to be revisited later. It accumulates over time and can significantly impact the quality, maintainability, and performance of the software. **Always** try to minimize technical debt!

4 Design Principles

EXAM ALERT — From Quiz 1 Answer Key

The professor tested design principles heavily in Quiz 1. The **Open-Closed Principle** and **Refactoring** were worth 1.5 marks. **SOLID** was accepted as an alternative answer. Know these very well!

4.1 SOLID Principles

SOLID — Five Design Principles

SOLID is a set of five design principles that help create **maintainable, flexible, and scalable** software:

Letter	Principle	Meaning
S	Single Responsibility	A class should have only one reason to change (one job).
O	Open-Closed	Software entities should be open for extension, closed for modification .
L	Liskov Substitution	Subtypes must be substitutable for their base types without breaking the program.
I	Interface Segregation	Clients should not be forced to depend on interfaces they don't use .
D	Dependency Inversion	Depend on abstractions , not on concrete implementations.

4.2 Open-Closed Principle (OCP) — In Detail

Open-Closed Principle (OCP) — CRITICAL FOR EXAM

“Software entities (classes, modules, functions) should be open for extension but closed for modification.”

- **Open for Extension:** You CAN add new functionality by **extending** existing components (e.g., adding new subclasses, implementing new interfaces).
- **Closed for Modification:** You should NOT need to **change existing, working code** to add new features.

Why it matters: When you modify existing code, you risk **breaking working functionality**. By extending instead of modifying, you keep the existing codebase stable.

Example: Instead of modifying a `PaymentProcessor` class every time a new payment method is added, create an abstract `Payment` interface and add new classes like `CreditCardPayment`, `UPIPayment`, etc.

4.3 GRASP Principles

GRASP — General Responsibility Assignment Software Patterns

GRASP provides guidelines for assigning **responsibilities** to classes in object-oriented design:

- **Information Expert:** Assign responsibility to the class that has the **information needed** to fulfill it
- **Creator:** Assign class B the responsibility to create an instance of class A if B contains, aggregates, or closely uses A
- **Low Coupling:** Minimize **dependencies** between classes
- **High Cohesion:** Keep related responsibilities **together** in one class
- **Controller:** Assign the responsibility of handling system events to a **controller class**

4.4 Coupling & Cohesion

Concept	Goal	Description
Coupling	Low (loose)	Degree of interdependence between modules. Low coupling = modules work independently.
Cohesion	High	Degree to which elements in a module belong together . High cohesion = focused, single-purpose modules.

Remember

Good design = **Low Coupling + High Cohesion**. Tightly coupled systems are **rigid** and **fragile** — changing one thing breaks other things (this was explicitly tested in Quiz 1 Set B!).

4.5 Types of Software Maintenance

Four Types of Software Maintenance

Type	Description
Corrective Maintenance	Fixing bugs and defects found after release.
Adaptive Maintenance	Modifying software to work in a changed environment (new OS, new hardware, new regulations).
Perfective Maintenance	Enhancing existing features and improving performance , usability, or efficiency based on user feedback.
Preventive Maintenance	Making changes to prevent future problems . Improving maintainability, refactoring, updating documentation.

Quiz 1 Crossword: PERFECTIVE MAINTENANCE

“Perfective Maintenance” was a crossword answer. Remember: it’s about **enhancing** features carefully, not just fixing bugs. Also note “Preventive Maintenance” was mentioned in Set C answers — stopping errors **before** they occur.

5 Agile Development

5.1 The Agile Manifesto (2001)

The Agile Manifesto — Four Core Values

We value:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

*That is, while there is value in the items on the right, we value the items on the left **more**.*

5.2 12 Agile Principles (Key Ones)

- Highest priority: **satisfy the customer** through early and continuous delivery of valuable software
- Welcome **changing requirements**, even late in development
- Deliver **working software frequently** (weeks rather than months)
- Business people and developers must **work together daily**
- Build projects around **motivated individuals**
- The most efficient method of conveying information is **face-to-face conversation**
- **Working software** is the primary measure of progress
- Promote **sustainable development** — constant pace
- Continuous attention to **technical excellence** and good design
- **Simplicity** — maximize the amount of work NOT done
- The best architectures emerge from **self-organizing teams**
- Team regularly **reflects** on how to become more effective

5.3 Scrum Framework

Scrum — The Most Popular Agile Framework

Key Roles:

- **Product Owner:** Defines features, prioritizes backlog
- **Scrum Master:** Facilitates the process, removes obstacles
- **Development Team:** Self-organizing, cross-functional (3–9 people)

Key Ceremonies:

- **Sprint Planning:** Plan what to do in the next sprint (2–4 weeks)
- **Daily Standup:** 15-min meeting — What did I do? What will I do? Any blockers?
- **Sprint Review:** Demo working software to stakeholders
- **Sprint Retrospective:** Reflect and improve for next sprint

Key Artifacts:

- **Product Backlog:** Ordered list of everything needed in the product
- **Sprint Backlog:** Items selected for the current sprint
- **Increment:** The working software delivered at the end of a sprint

5.4 User Stories

User Story

A **natural-language description** of a software feature from the **user’s perspective**.

Template: “As a [type of user], I want [some goal] so that [some reason].”

Example: “As a *customer*, I want to *search for books by title* so that *I can find the book I want to buy*.”

5.5 Key Agile Practices

Practice	Description
Refactoring	Restructuring existing code to improve readability, maintainability, and efficiency without changing its behavior .
Test-Driven Development (TDD)	Write automated tests first , then write the code to make the tests pass. Cycle: Red → Green → Refactor.
Continuous Integration (CI)	Integrate code changes into a central repository frequently (multiple times a day). Automated builds and tests run on each integration.
Continuous Deployment (CD)	Automatically deploy validated code changes to production.
Pair Programming	Two developers work together at one workstation. One writes code (driver), the other reviews (navigator).

5.6 DevOps

DevOps

DevOps = **D**evelopment + **O**perations. It is a set of practices that combines software development and IT operations to **shorten the development lifecycle** and deliver software **continuously** with high quality. Key goal: minimize variability of estimates and improve project outcomes.

5.7 Agile Process Spectrum

End	Description
Plan-Driven (Rigid)	Waterfall: everything planned upfront, sequential
Middle Ground	Spiral, Incremental: some flexibility
Agile (Flexible)	Scrum, Kanban, XP: iterative, adaptive, respond to change

6 Project Estimation & Scheduling

6.1 The Iron Triangle of Project Management

Three Constraints for Project Success

A project is considered successful when it is delivered:

1. **On Time** — Meeting the deadline
2. **Within Budget** — Not exceeding the allocated cost
3. **According to Specifications** — Meeting all requirements

Only about 10–20% of projects achieve all three!

6.2 Function Point Analysis (FPA)

Function Point Analysis

A method of estimating the **size** of a software project by measuring its **functionality**. It categorizes functions into five types:

Function Type	Description
External Inputs (EI)	Data entering the system (e.g., forms, files)
External Outputs (EO)	Data leaving the system (e.g., reports, screens)
External Inquiries (EQ)	Input/output combinations (e.g., search queries)
Internal Logical Files (ILF)	Data maintained inside the system (e.g., database tables)
External Interface Files (EIF)	Data referenced from external systems

Steps:

1. Identify all functions in each category
2. Assign **complexity weights** (Simple, Average, Complex) to each
3. Calculate **Unadjusted Function Points (UFP)** = $\sum(\text{count} \times \text{weight})$
4. Apply **Value Adjustment Factor (VAF)** based on 14 general system characteristics
5. **Adjusted Function Points** = $\text{UFP} \times \text{VAF}$
6. **Effort** = $\text{Adjusted FP} \times \text{Effort per FP}$

FPA Example: Online Bookstore

Function	Type	Complexity	Weight	FP
User Registration	EI	Average	4	4
Product Listing	EO	Complex	7	7
Search Books	EQ	Average	4	4
Book Database	ILF	Complex	15	15
Payment Gateway	EIF	Simple	5	5
Total UFP				35

6.3 Value Adjustment Factor (VAF)**VAF Calculation**

Based on **14 General System Characteristics (GSC)**, each rated 0–5:

- | | |
|--------------------------------|-----------------------|
| 1. Data communications | 8. Online update |
| 2. Distributed data processing | 9. Complex processing |
| 3. Performance | 10. Reusability |
| 4. Heavily used configuration | 11. Installation ease |
| 5. Transaction rate | 12. Operational ease |
| 6. Online data entry | 13. Multiple sites |
| 7. End-user efficiency | 14. Facilitate change |

$$\text{VAF} = 0.65 + 0.01 \times \sum_{i=1}^{14} \text{GSC}_i$$

The VAF ranges from **0.65** (when all GSCs = 0) to **1.35** (when all GSCs = 5).

6.4 Work Breakdown Structure (WBS)**Work Breakdown Structure**

A **hierarchical decomposition** of the total scope of work into smaller, manageable tasks. Each level breaks down the work further until tasks are small enough to be estimated and assigned.

Example:

- Project: Online Bookstore
 - Phase: Design
 - * Task: Design database schema
 - * Task: Design UI wireframes
 - Phase: Implementation

- * Task: Implement user registration
- * Task: Implement search functionality

6.5 Gantt Charts & Critical Path

Gantt Chart

A **bar chart** that shows the project schedule visually. Each bar represents a task, showing its **start date**, **end date**, **duration**, and **dependencies** with other tasks.

Critical Path

The **longest sequence of dependent tasks** that determines the **minimum project duration**. Any delay in the critical path delays the **entire project**. Non-critical tasks have **slack time** (can be delayed without affecting the project end date).

Slack Time (Float)

Slack time = the amount of time a task can be **delayed** without delaying the project's end date.

$$\text{Slack Time} = \text{Latest Start} - \text{Earliest Start}$$

Key Rules:

- Tasks **on the critical path** have **zero slack time**
- Tasks **NOT on the critical path** have **positive slack time**

EXAM-STYLE Worked Example: Critical Path & Slack Time

This is modeled directly on Quiz 1, Question 2.

Consider a project with the following tasks and dependencies:

Task	Duration (hrs)	Task	Duration (hrs)
A	15	F	20
B	20	G	25
C	15	H	10
D	20	I	7
E	28		

Dependencies: A→B, A→C, B→E, C→D, E→F, D→G, F→G, G→I, F→H

Step 1: Identify all paths and their durations:

- Path 1: A → B → E → F → G → I = 15+20+28+20+25+7 = **115 hrs**
- Path 2: A → C → D → G → I = 15+15+20+25+7 = **82 hrs**
- Path 3: A → B → E → F → H = 15+20+28+20+10 = **93 hrs**

Step 2: Critical Path = the **longest path** = A → B → E → F → G → I = **115 hours**

Step 3: Slack Time for D:

- Critical path takes 115 hours total
- Path $A \rightarrow C \rightarrow D$ merges at G. After D, the path is $D \rightarrow G \rightarrow I = 25 + 7 = 32$ hrs
- Path from A to just before G via C,D: $A + C + D = 15 + 15 + 20 = 50$ hrs
- Available time from start to G via this path: $115 - 32 = 83$ hrs
- Slack for entire sub-path $A \rightarrow C \rightarrow D$: $83 - 50 = 33$ hrs BUT A is shared (on critical path), so:
- Available time for D = Total critical path – (A+C before D) – (G+I after D) = $115 - 30 - 32 = 53$
- D takes 20 hrs. Slack for D = $53 - 20 = 33 - 20 = 13$ hours (using the path comparison method)

Step 4: Slack Time for E:

- E is on the critical path, so Slack Time = 0 hours

6.6 Earned Value Management (EVM)

Earned Value Management

A method for measuring project progress by comparing:

- **Planned Value (PV):** The value of work planned to be done by now
- **Earned Value (EV):** The value of work actually completed by now
- **Actual Cost (AC):** What has actually been spent so far

Key Metrics:

- **Schedule Variance** = $EV - PV$ (positive = ahead; negative = behind)
- **Cost Variance** = $EV - AC$ (positive = under budget; negative = over budget)
- **Schedule Performance Index (SPI)** = EV / PV
- **Cost Performance Index (CPI)** = EV / AC

7 Requirements Engineering

What is Requirements Engineering?

The process of **gathering, analyzing, documenting, and maintaining** the requirements of a software system. It ensures the software meets the needs and expectations of its stakeholders. **Poor requirements gathering is the #1 reason projects fail!**

7.1 Functional vs. Non-Functional Requirements

Aspect	Functional Requirements	Non-Functional Requirements
Definition	What the system should do	How the system should behave
Focus	Specific functions/features	Quality attributes & constraints
Examples	“User can log in with username & password” “User can search for books by title” “System processes credit card payments”	“System responds within 2 seconds” “System supports 1000 concurrent users” “Data encrypted with AES-256”

7.2 Requirements Gathering Techniques

Technique	Description
Interviews	One-on-one conversations with stakeholders
Questionnaires/Surveys	Structured forms distributed to many stakeholders
Observation	Watching users perform their tasks in their environment
Brainstorming	Group sessions to generate ideas
Prototyping	Building a mock-up to validate requirements
Use Case Analysis	Modeling user interactions with the system
Workshops	Collaborative sessions with stakeholders and developers

7.3 Qualities of Good Requirements

A Good Requirement Must Be:

Quality	Meaning
Correct	Accurately reflects the stakeholder’s needs
Complete	Covers all necessary aspects; nothing is missing
Unambiguous	Can only be interpreted one way
Consistent	Does not contradict other requirements
Feasible	Can be implemented with available resources and technology
Verifiable	Can be tested to confirm it is met
Traceable	Can be linked back to its origin and forward to design/test
Concise	Short and to the point; no unnecessary words

7.4 Categorizing Requirements: Basic, Differentiators, Delights

Kano Model for Features

Category	Description
Basic (Must-Haves)	Essential features. Users assume these exist. Absence causes dissatisfaction, but presence doesn't increase satisfaction.
Differentiators	Features that distinguish your product from competitors. More = more satisfaction.
Delights	Unexpected features that exceed expectations. The “wow” factor.

Evolution: Over time, Delights → Differentiators → Basic. What was once a delight becomes the new basic expectation.

7.5 MoSCoW Prioritization

MoSCoW Method

A prioritization technique:

- **Must-Have:** Critical requirements; system doesn't work without them
- **Should-Have:** Important but not critical; can work around temporarily
- **Could-Have:** Nice to have; included if time/budget permits
- **Won't-Have (this time):** Acknowledged but deferred to a future release

7.6 Minimum Viable Product (MVP)

MVP

The **Minimum Viable Product** is a version of the product with the **smallest set of features** sufficient to satisfy early customers and gather feedback for future development. It reduces risk and validates market assumptions.

7.7 Verification vs. Validation

V&V — Know the Difference!

	Verification	Validation
Question	“Are we building the product right ?”	“Are we building the right product?”
Focus	Conformance to specifications	Meeting user needs
Activity	Reviews, inspections, walkthroughs	Testing with actual users

7.8 Software Requirements Specification (SRS)

SRS Document

The SRS is a **living document** that serves as a contract between developers and customers. It contains:

- Functional requirements
- Non-functional requirements
- Constraints
- Assumptions and dependencies
- Acceptance criteria

Important: The SRS may change throughout development as new information becomes available. Requirements analysis is an **iterative process**.

7.9 Data Privacy & Regulations

Data Privacy in Requirements

Software systems must comply with data privacy regulations:

- **GDPR** (General Data Protection Regulation) — Europe
- **India's Data Protection Act**
- Consider encryption, access controls, data anonymization
- These become **non-functional requirements** that must be addressed early!

8 Use Case Modeling

What is a Use Case?

A **use case** is a description of a **sequence of actions** that a system performs in response to an interaction initiated by an **actor**, resulting in a **specific outcome** of value. It describes the system's behavior from the **user's perspective** (the system is treated as a **black box**).

8.1 Actors

Actor

An **actor** is an external entity (person, system, or device) that interacts with the system.

Actor Type	Description
Primary Actor	Initiates the interaction with the system (e.g., Customer, Cashier)
Supporting (Secondary) Actor	Provides services or information needed by the system (e.g., Payment Gateway, Tax Calculator, Bank System)

Note: Actors are NOT limited to humans! They can be other systems, time-based triggers (e.g., a timer), or any external entity.

8.2 Use Case Template

Standard Use Case Template

Field	Description
Use Case Name	Short, descriptive name (verb + noun, e.g., "Withdraw Cash")
Brief Description	One-sentence summary of the use case
Type	Primary / Secondary
Importance	High / Medium / Low
Primary Actor	Who initiates the interaction
Supporting Actors	Other actors involved
Preconditions	What must be true BEFORE the use case starts
Main Flow	Step-by-step description of the normal (happy) path
Alternate Flows	Variations or exceptions to the main flow
Post-conditions	What is true AFTER the use case completes successfully
Non-functional Properties	Performance, security constraints (optional)

Example: ATM — Withdraw Cash

Use Case Name	Withdraw Cash
Primary Actor	Customer
Supporting Actor	Bank System
Precondition	Customer has a valid ATM card and the ATM is operational
Main Flow	1. Customer inserts ATM card 2. System prompts for PIN 3. Customer enters PIN 4. System verifies PIN with Bank System 5. System displays menu options 6. Customer selects “Withdraw Cash” 7. Customer enters amount 8. System checks balance with Bank System 9. System dispenses cash 10. System prints receipt
Alternate Flow	4a. PIN is incorrect → System prompts to retry (max 3 times) 8a. Insufficient funds → System displays error message
Post-condition	Customer has received cash; account balance is updated

8.3 Use Case Relationships

Relationship	Description
Include (⟨⟨include⟩⟩)	Use case A always uses use case B. B is a mandatory sub-behavior. Example: “Process Payment” includes “Validate Card”.
Extend (⟨⟨extend⟩⟩)	Use case B optionally extends use case A. B adds behavior under certain conditions. Example: “Checkout” may be extended by “Apply Coupon”.
Generalization	Use case B is a specialized version of use case A. Example: “Pay by Credit Card” and “Pay by PayPal” are specializations of “Make Payment”.

8.4 Granularity of Use Cases

Finding the Right Level of Detail

- Too coarse: “Use the system” — this is useless
- Too fine: “Click the button” — this is too detailed
- **Right level:** Each use case should represent a **complete, meaningful interaction** that delivers value to an actor
- Think of it as: “What can the actor accomplish in one sitting?”

9 Conceptual Modeling & UML

What is a Conceptual Model?

A **conceptual model** (also called a **domain model**) is a high-level representation of the **key concepts, attributes, and relationships** in a system. It is created by analyzing the use case model and identifies the “nouns” in the system.

It helps bridge the gap between **requirements** (problem space) and **design** (solution space).

9.1 From Use Cases to Conceptual Model

Step-by-Step Process

1. Read through all use case descriptions
2. **Extract nouns and noun phrases** — these are candidate **conceptual classes**
3. Filter out duplicates, irrelevant terms, and attributes
4. Identify **attributes** for each class (primitive data types: string, int, date, etc.)
5. Identify **relationships (associations)** between classes
6. Define **multiplicity** for each relationship
7. Draw the conceptual model using **UML class diagram notation**

9.2 UML Class Diagram Notation

UML (Unified Modeling Language)

UML is the **industry-standard** modeling language for representing software systems. For conceptual models, we use **class diagrams**.

A **class box** has three **compartments**:

ClassName
– attribute1: Type – attribute2: Type
+ method1(): ReturnType + method2(param: Type): ReturnType

9.3 Access Modifiers

Symbol	Modifier	Meaning
+	Public	Accessible from anywhere
–	Private	Accessible only within the class
#	Protected	Accessible within the class and its subclasses

Key Rule

Attributes should be **private** (–). Access to them should be controlled through **public methods** (getters/setters). This is called **encapsulation**.

9.4 Associations & Multiplicity

Association

An **association** represents a **relationship** between two conceptual classes.

Multiplicity defines how many instances of one class can be associated with instances of another class.

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one (optional)
* or 0..*	Zero or more
1..*	One or more
n..m	Between n and m

Example: Point-of-Sale System

Key conceptual classes and relationships:

- **Sale** — contains — **SaleLineItem** (1..*)
- **SaleLineItem** — describes — **Product** (1)
- **Sale** — paid-by — **Payment** (1)
- **Store** — stocks — **Product** (*)
- **Cashier** — initiates — **Sale** (*)

9.5 Types of Attributes

Type	Description
Primitive	Basic data types: <code>int</code> , <code>String</code> , <code>boolean</code> , <code>Date</code> , <code>double</code>
Derived	Calculated from other attributes (e.g., <code>totalPrice = quantity × unitPrice</code>)

9.6 Key Distinction: Conceptual vs. Software Classes

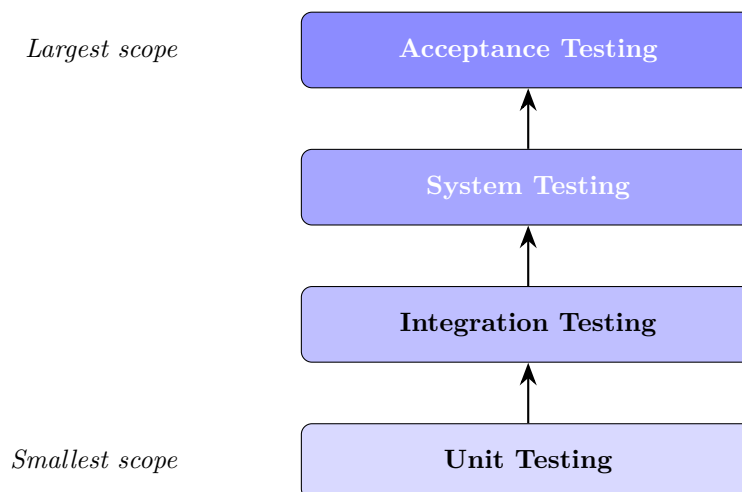
Aspect	Conceptual Class	Software Class
Level	High-level; domain concepts	Implementation-level
Purpose	Understand the problem	Solve the problem in code
Detail	No methods, just concepts	Full methods, data structures
Phase	Analysis	Design/Implementation

10 Software Testing

What is Testing?

Testing is the process of **evaluating a system or component** under **specified conditions**, **observing the results**, and making an **evaluation** of some aspect of the system. Testing is NOT just about finding bugs — it's about ensuring the system **meets requirements and standards**.

10.1 Levels of Testing



Level	Description
Unit Testing	Testing individual units of code (functions, methods, classes). Each unit is tested independently . Tests should cover all scenarios and edge cases.
Integration Testing	Testing how combined units work together . Focus is on the interfaces and communication between modules, NOT re-testing individual units.
System Testing	Testing the complete, integrated system against requirements. Performed in the real-world environment . Tests functionality, performance, security, usability.
Acceptance Testing	Testing by end-users/stakeholders to ensure the system meets their requirements and expectations. Final sign-off before release.

10.2 Integration Testing Strategies

Strategy	Description
Big Bang	Integrate all units at once and test. Simple but hard to isolate errors.
Top-Down	Start from top-level modules and integrate downward. Uses stubs for lower modules not yet ready.
Bottom-Up	Start from lowest-level modules and integrate upward. Uses drivers for upper modules not yet ready.
Sandwich / Hybrid	Combines top-down and bottom-up. Test from both ends toward the middle.

10.3 Black Box vs. White Box Testing

Aspect	Black Box Testing	White Box Testing
Knowledge	No knowledge of internal code	Full knowledge of internal code
Focus	Input/Output behavior	Internal logic & structure
Techniques	Equivalence partitioning, boundary value analysis	Statement coverage, branch coverage, path coverage
Who does it?	Usually testers	Usually developers
Also called	Functional testing	Structural testing

10.4 Black Box Testing Techniques

10.4.1 Equivalence Partitioning

Equivalence Partitioning

Divide the input data into **partitions (classes)** where all values in a partition are expected to be **treated the same way** by the system. Test **one value from each partition** instead of testing all possible values.

Example: For a field accepting ages 18–65:

- **Valid partition:** 18–65 (pick any value, e.g., 30)
- **Invalid partition 1:** < 18 (e.g., 10)
- **Invalid partition 2:** > 65 (e.g., 80)

10.4.2 Boundary Value Analysis

Boundary Value Analysis (BVA)

Test at the **boundaries (edges)** of input ranges, because errors are most likely to occur there.

Example: For the range 18–65, test:

- **Just below minimum:** 17
- **At minimum:** 18
- **Just above minimum:** 19

- Just below maximum: 64
- At maximum: 65
- Just above maximum: 66

10.4.3 State Transition Testing

State Transition Testing

Tests the system's behavior as it **transitions between states**. Useful for systems with finite states (e.g., login attempts, ATM states).

Example: ATM PIN entry: Idle $\xrightarrow{\text{insert card}}$ Waiting for PIN $\xrightarrow{\text{correct PIN}}$ Authenticated
 $\xrightarrow{\text{wrong PIN (3rd time)}}$ Card Blocked

10.5 White Box Testing Techniques

Coverage Type	Description
Statement Coverage	Every statement in the code is executed at least once
Branch/Decision Coverage	Every branch (true/false) of each decision point is executed
Condition Coverage	Every individual condition in every decision is tested as both true and false
Path Coverage	Every possible execution path through the code is tested (most thorough but often impractical)

10.6 Test Case

Test Case

A **test case** is a set of:

- **Preconditions:** What must be true before the test
- **Inputs:** The data fed to the system
- **Expected Results:** What the system should produce

Test cases are developed to drive the execution of a test item and to meet specific test objectives.

10.7 Types of Acceptance Testing

Type	Description
Alpha Testing	Done in-house by the development team or internal users
Beta Testing	Released to a limited external audience for real-world feedback
Crowd Testing	Large, diverse group of external testers
Compliance Testing	Ensures the system meets regulatory requirements and standards

10.8 Key Testing Principles

Seven Testing Principles to Remember

1. **Testing shows presence of defects**, not their absence
2. **Exhaustive testing is impossible** (too many combinations)
3. **Early testing** saves time and money
4. **Defects cluster together** (Pareto principle: 80% of bugs in 20% of code)
5. **Pesticide paradox**: Running same tests repeatedly won't find new bugs
6. **Testing is context-dependent** (different for safety-critical vs. e-commerce)
7. **Absence-of-errors fallacy**: Bug-free software is useless if it doesn't meet requirements

10.9 Testing Metrics

Metric	Description
Code Coverage	Percentage of code executed during testing
Defect Density	Number of defects per unit of code (e.g., per 1000 LOC)
Defect Detection Rate	Percentage of defects found before release
Test Pass Rate	Percentage of test cases that pass successfully

10.10 Qualities of a Good Tester

Qualities of a Good Tester

- Independent
- Curious
- Patient
- Courageous
- Detail-oriented
- Good communicator
- Good at documentation
- Systematic thinker

10.11 Regression Testing & Maintenance Testing

Regression Testing

Re-testing **existing functionality** after changes/updates to ensure that previously working features **still work correctly**. Run after every code change.

Maintenance Testing

Testing performed **after release** to ensure the system continues to meet requirements as it is updated, modified, and repaired over its lifetime.

11 Introduction to Design

What is Software Design?

Design is the activity of creating a detailed plan that translates requirements into a feasible technical solution. It is not just about aesthetics — it is about **solving problems** by deciding system structure, component responsibilities, and interactions. Good design **reduces the number of potential defects**, directly lowering testing and debugging effort.

11.1 Design vs. Coding: A Critical Distinction

The Professor’s Message (March 9)

“Understanding systems and architecture is crucial for a career in software development.” Merely generating code is not sufficient. In industry, most work is on **existing** codebases (brown-field), so being able to **communicate** about systems using models is vital.

- **Writing code** verifies a specification. **Modeling** is about understanding system requirements and behavior before code exists.
- Design is **language-agnostic** — a good design does not depend on any specific programming language.
- Design is **iterative**: it involves refining through multiple cycles of analysis, alternatives, and evaluation.
- Design involves **trade-offs** and considering multiple stakeholder viewpoints, not picking a single “right” answer.

11.2 Models and Modeling Languages

Model vs. Object

A **model** is an abstraction of the real thing — it deliberately omits detail to focus on what matters. When we communicate about a system, we use models (representations), not the objects themselves.

Language Type	Description & Examples
Declarative	Specify <i>what</i> to accomplish (not how). Example: SQL.
Imperative	Specify <i>how</i> to accomplish it as a sequence of steps. Example: Python, Java.
Domain-Specific Language (DSL)	Constrained vocabulary tailored to a domain. Examples: MATLAB (matrix ops), HTML (markup), SQL (queries).
General-Purpose Language (GPL)	Broad vocabulary, applicable across domains (Java, Python).

UML — The De-facto Modeling Standard

Unified Modeling Language (UML) is the industry-standard graphical notation for software modeling. Key diagram families:

- **Structural:** Class diagram, Object diagram, Component diagram, Deployment diagram.
- **Behavioral:** Use-case diagram, Sequence diagram, State machine, Activity diagram.

UML can also model business processes, not just software.

11.3 Core Design Principles

11.3.1 Abstraction

Abstraction

Conveying what is **necessary** and hiding what is **not necessary**. It reduces complexity so designers can focus on essentials.

- **Data abstraction:** Focus on what data is held, not how it is stored (e.g., stack ADT hides array-vs-linked-list).
- **Procedural abstraction:** Focus on inputs and outputs of a function, not its internal steps.
- **Control abstraction:** Handling exceptions and flow without exposing internal mechanics.

11.3.2 Separation of Concerns

Divide the system into distinct sections, each addressing a **specific concern** (UI, business logic, data storage). This reduces complexity, improves maintainability, and enhances scalability.

11.3.3 Modularity

A system is **modular** when it is composed of independent modules that can be modified, replaced, or exchanged **without affecting** the rest of the system.

11.3.4 Goal: Loose Coupling + High Cohesion

- **Coupling** = degree of interdependence *between* modules. Aim: **low**.
- **Cohesion** = degree of unity *within* a module. Aim: **high**.

11.4 Design Smells

Design Smell

A characteristic of a design that **indicates a deeper problem**, though the system still functions. Smells suggest the design is fragile, rigid, or hard to change.

Smell	Meaning
God Object	A class that knows or does too much.
Shotgun Surgery	A single change forces edits in many different classes.
Long Method	A method with far too many responsibilities.
Duplicate Code	The same logic appears in multiple places.
Feature Envy	One class uses methods of another far more than its own.

Design Smell vs. Design Pattern

A **design smell** is a *symptom* of a poor design. A **design pattern** is a *proven solution* to a recurring design problem. Smells tell you *what is wrong*; patterns tell you *how to fix it or avoid it*.

11.5 Practice & Review — Introduction to Design

Exam-Style Questions (March 5 & 9)

Q1 (MCQ). What is the primary purpose of abstraction in system design?

Ans: To simplify complex systems by showing only the necessary information.

Q2 (Short Answer). Differentiate declarative and imperative languages. Give one example of each.

Ans: Declarative specifies *what* (SQL); imperative specifies *how* (Python/Java).

Q3 (Essay). Explain separation of concerns. How does it relate to modularity and cohesion?

Ans: Separation of concerns divides a system into independent sections, each with a single responsibility. This directly creates **high cohesion** within modules (each focused) and **low coupling** between them (minimal shared state), making the system easier to change, test, and scale.

Q4 (Problem-Solving). A system has tightly coupled components. Describe a refactoring strategy to achieve low coupling and high cohesion.

Ans: Identify the responsibilities currently mixed; extract them into cohesive classes; introduce interfaces between them; invert dependencies so high-level modules depend on abstractions; add tests around behaviour before changing structure.

Q5 (MCQ). What is the primary purpose of a Domain-Specific Language (DSL)?

Ans: To offer a language tailored for a specific application domain with a constrained, expressive vocabulary.

Q6 (Short Answer). Distinguish a model from a meta-model.

Ans: A model is an *instance* of a meta-model; the meta-model defines the structural rules (syntax and constraints) to which all conforming models must adhere.

12 Cybersecurity and Secure Design

Secure by Design

Secure by Design is the principle that security should be **built into a system from the very beginning** of the design process, rather than bolted on as an afterthought. Security is not merely a technical issue — it is a **design and architectural** concern. Systems that are not secure by design accumulate security debt that becomes increasingly expensive to remediate.

Guest Speaker Context — March 9

This lecture was delivered by a **Chief Product Officer** with extensive industry experience in financial systems. The talk grounded abstract security concepts in real-world systems: a World Bank cash-transaction processing system and a pharmacy insurance claims processor. Both examples demonstrated how security oversights at the design phase lead to catastrophic consequences at scale.

12.1 Non-Functional Requirements and Security

Why Non-Functional Requirements Are Not Optional

Functional requirements describe *what* a system does. Non-functional requirements (NFRs) describe *how well* it does it. NFRs such as **security, performance, scalability, reliability, and usability** are equally important — a system that functions correctly but is insecure, slow, or unusable is a failed system.

NFR Category	Examples & Design Implications
Security	Authentication, authorisation, encryption, audit logs, zero-trust validation.
Performance	Throughput, latency (e.g., pharmacy claim processed in <3 seconds).
Scalability	Horizontal vs. vertical scaling; database normalisation to handle volume.
Reliability / Availability	Fault tolerance, MTBF, disaster recovery, redundancy.
Usability	Learnability, accessibility, error recovery.

12.2 Zero Trust Security Model

Zero Trust

Zero Trust is a security paradigm based on the principle: “*never trust, always verify.*” No user, device, or network segment is trusted by default — every request must be **authenticated, authorised, and continuously validated**, regardless of origin (inside or outside the network perimeter).

Core Zero Trust mechanisms:

- **Multi-Factor Authentication (MFA):** Combine something you know (password), have (token), and are (biometric).

- **Least Privilege Access:** Users and processes get the minimum permissions needed — nothing more.
- **Micro-segmentation:** Divide the network into small zones so that a breach in one zone cannot propagate freely.
- **Continuous Monitoring:** Log and audit every action; treat anomalies as potential threats.

Zero Trust vs. Traditional Perimeter Security

Traditional security trusts everything inside the firewall. Zero Trust treats the **internal network as untrusted** — it defends against insider threats and lateral movement after a breach, which is where most damage occurs.

12.3 Cybersecurity Risk Management

Cybersecurity is fundamentally a **risk management discipline**. The goal is not to eliminate all risk (impossible), but to **reduce risk to an acceptable level** given available resources.

Concept	Explanation
Threat	A potential event that could harm the system (e.g., data breach, insider fraud).
Vulnerability	A weakness that a threat can exploit (e.g., SQL injection, unpatched library).
Risk	Likelihood × Impact. Risk drives the priority of mitigation.
Control / Countermeasure	A safeguard that reduces likelihood or impact (e.g., firewall, encryption).
Residual Risk	Risk remaining after controls are applied — accepted by management.

12.4 Security Testing

- **Penetration Testing:** Ethical hackers attempt to exploit the system to discover real vulnerabilities before attackers do.
- **Vulnerability Scanning:** Automated tools scan for known weaknesses in code, dependencies, and configuration.
- **Functional Security Testing:** Verify that authorisation logic is correct — does User A get access to User B's data?
- **Non-Functional Security Testing:** Load test the authentication stack; verify the system does not degrade under attack traffic.
- **Insider Threat Testing:** Simulate a malicious or compromised internal user — particularly important in financial systems.

12.5 Case Study — World Bank: Cash Transaction Processing

The Salami Attack (Penny Shaving)

When the guest speaker was building a multi-currency transaction processing system for the World Bank, a critical design concern arose: **rounding errors in currency conversions**.

The attack: In a system handling millions of daily cross-currency transactions, even a fraction-of-a-cent rounding difference per transaction accumulates to an enormous sum. An adversary (or insider) who systematically routes those fractions to a controlled account commits a **Salami Attack** — slicing off imperceptibly small amounts from thousands of transactions.

Design responses:

- Use **exact-arithmetic** libraries (not floating-point) for financial calculations.
- Apply **round-half-to-even** (banker's rounding) — statistically unbiased over many transactions.
- Maintain **audit trails** of every rounding decision.
- Run **reconciliation checks**: total debits must equal total credits within machine epsilon.

Financial Messaging Standards

- **SWIFT**: Society for Worldwide Interbank Financial Telecommunication — the global standard for secure inter-bank messages.
- **SFMS**: Structured Financial Messaging System — India's domestic equivalent of SWIFT.
- **UPI**: Unified Payments Interface — India's real-time gross settlement retail payment system. Enables instant fund transfer between banks.

12.6 Case Study — Pharmacy Insurance Claims System

Pharmacy Claims Under Extreme NFR Pressure

The system needed to:

1. Verify patient insurance coverage.
2. Check for dangerous drug interactions with the patient's medication history.
3. Approve or reject the claim — all within **3 seconds**.

Security challenges: Patient health data is *extremely* sensitive (HIPAA/healthcare regulations). All data in transit must be encrypted; at rest must be access-controlled by role. Insider threats (pharmacy staff accessing records they should not) must be audited and alerted.

Performance challenge: A 3-second SLA with simultaneous nationwide requests requires careful database design, query optimisation, and infrastructure scaling — which

is why database normalisation and index design are **security-adjacent** concerns (a slow system is an availability attack surface).

12.7 Database Normalization and Data Integrity

Database Normalization

The process of organising data to **minimise redundancy and dependency**. Normalisation rules (Normal Forms) ensure that each piece of data is stored in exactly one place, reducing inconsistency and improving scalability.

- **1NF (First Normal Form):** Each table cell contains a *single*, atomic value. No repeating groups.
- **2NF:** 1NF + all non-key attributes are *fully* dependent on the primary key (no partial dependencies).
- **3NF:** 2NF + no transitive dependencies (non-key attributes depend only on the primary key, not on each other).

Why this matters for security: Redundant data creates **inconsistency** — a data anomaly can mask fraud or create privilege-escalation paths (e.g., duplicate user records with different permission levels).

12.8 Network Security Fundamentals

Component	Role
Router	Directs data packets between networks using IP addresses.
Firewall	Monitors and filters incoming/outgoing traffic based on security rules. Blocks unauthorized traffic.
Switch	Connects devices within the same network; acts as a traffic controller at the local level.
DMZ (Demilitarised Zone)	A network segment between the internal trusted network and the external internet, hosting public-facing servers.
VPN	Encrypts traffic over a public network, creating a secure tunnel.

12.9 Open-Source Software Security

Open-source software (OSS) has a **dual-nature security profile**:

Advantage	Explanation
Community oversight	Many independent reviewers can find and fix vulnerabilities quickly (<i>Linus's Law</i> : “given enough eyeballs, all bugs are shallow”).
Transparency	Source code is inspectable — no hidden backdoors from vendors.
Risk	Explanation
Anyone can contribute	A malicious contributor can introduce a backdoor (supply-chain attack).
Unvetted dependencies	A vulnerability in a transitive dependency can compromise the entire system.
Lack of centralized support	When a maintainer abandons a project, vulnerabilities may go unpatched.

Domain Knowledge is a Security Prerequisite

The guest speaker emphasised that **domain expertise** (banking, insurance, healthcare) is indispensable for building secure systems. A developer who does not understand the *business context* of a financial transaction cannot identify what a Salami Attack looks like, what data privacy regulations apply, or which operations are high-risk. Security decisions require understanding what the system *means*, not just how it *runs*.

12.10 Secure Coding Practices

- **Input Validation:** Validate all external input at system boundaries (user inputs, API responses, file uploads).
- **Design by Contract:** Specify preconditions, postconditions, and invariants — any violation signals a security or logic error.
- **Principle of Least Privilege:** Code runs with only the permissions it needs; database accounts have only the necessary operations.
- **Fail Securely:** When an error occurs, deny access by default rather than granting it.
- **Avoid Security by Obscurity:** Do not rely on hidden algorithms or data formats for security. Assume the attacker knows the design.

12.11 Digital Public Infrastructure

Digital Public Infrastructure (DPI)

The foundational digital systems — networks, platforms, payment rails, identity systems — that governments and economies build as shared public goods. Examples: UPI (India), Aadhaar, CoWIN (COVID vaccination). DPI must be designed for **extreme scale, security, inclusion, and interoperability**.

12.12 Practice & Review — Cybersecurity and Secure Design

Exam-Style Questions (March 9 Guest Lecture)

Q1 (MCQ). What does the “Zero Trust” security model require?

Ans: Verification of *every* user, device, and request, regardless of network location. No entity is trusted by default.

Q2 (MCQ). What is the primary purpose of a firewall?

Ans: To filter and block incoming/outgoing network traffic based on security rules.

Q3 (Short Answer). Differentiate functional and non-functional requirements. Give two examples of non-functional requirements critical for a banking system.

Ans: Functional requirements describe *what* a system does (process a transaction); non-functional describe *how well* (process in <1s, encrypt all data at rest). Banking NFRs: security (zero trust, encryption) and availability (99.999% uptime SLA).

Q4 (Short Answer). Explain the Salami Attack. What design decision can prevent it?

Ans: A Salami Attack steals tiny rounding errors from thousands of transactions, accumulating them into a significant sum. Prevented by using exact-arithmetic libraries instead of floating-point arithmetic, and by reconciliation checks that verify total debits = total credits.

Q5 (Essay). Explain “Secure by Design” and contrast it with “Security as an Afterthought.” Why is the latter more expensive?

Ans: Secure by Design integrates security into requirements, architecture, and implementation from day one. Security as an afterthought bolts security controls onto a finished system, which often requires rearchitecting core data flows, re-testing the entire system, and patching systemic vulnerabilities. Fixing a security flaw after deployment costs 10–100× more than catching it at design time (Boehm’s curve).

Q6 (Essay). Discuss the security implications of open-source software. Is OSS more or less secure than proprietary software?

Ans: OSS can be more secure because community review finds bugs quickly (Linus’s Law), and the code is inspectable for backdoors. It can be less secure due to supply-chain attacks, abandoned maintainers, and unvetted transitive dependencies. Neither model is universally superior — the key is the size and quality of the community, the criticality of the component, and the organisation’s due-diligence process for evaluating dependencies.

Q7 (Short Answer). What are the three steps of cybersecurity risk management?

Ans: (1) **Identify** threats and vulnerabilities; (2) **Assess** risk = likelihood × impact; (3) **Mitigate** by applying controls to reduce risk to an acceptable residual level.

13 UI/UX Design and Usability

Why Non-Functional Matters

When designing a system, **functionality is taken for granted**. What truly **differentiates** a solution is how well it meets *non-functional* requirements — usability, performance, security, maintainability, reliability.

13.1 UI vs. UX: A Clear Distinction

Term	Meaning
UI (User Interface)	The visual/interactive elements: layout, buttons, menus, aesthetics, spatial organisation.
UX (User Experience)	The <i>entire</i> experience of using the system — usability, accessibility, aesthetics, performance, emotional response. UI is a subset of UX.

Common Mistake

Saying “UI/UX” as if they are interchangeable loses marks. UI is what the user *sees and clicks*; UX is how the system *feels to use* in its entirety. A system can have a beautiful UI but terrible UX (slow, confusing, inaccessible).

13.2 Trade-offs in Design

Design is the art of making **conscious, justified trade-offs**. Every non-functional requirement competes with others.

Trade-off	Discussion
Security vs. Usability	More password rules, MFA, and lockouts frustrate users but protect them. Class example: <i>IMS</i> (strong security, painful UI).
Performance vs. Maintainability	Hand-tuned fast code is often hard to read. Readable code is sometimes slower.
Privacy vs. Functionality	Personalisation requires data; data use raises privacy concerns.
Consistency vs. Flexibility	A rigid consistent UI is predictable; a flexible UI adapts but may confuse.

13.3 Designing for the User

13.3.1 Personas

Fictional but evidence-based profiles of target users (name, demographics, goals, frustrations, tech literacy). Personas **anchor design decisions** in real user needs rather than the developer’s own assumptions.

13.3.2 Mental Models & Intuition

Mental Model

The user's internal understanding of *how the system works*. A system is **intuitive** when its UI aligns with the user's existing mental model — the edit distance between what the user *expects* and what the system *does* is small.

13.3.3 Cognitive Load

The mental effort required to use the system. Good design **minimises cognitive load**: clear labelling, progressive disclosure, defaults, short paths to common actions.

13.4 Information Architecture

The **way information is organised and flows** across the system from the user's perspective — navigation, grouping, labelling, hierarchy. Poor IA makes even a good UI unusable; good IA lets users find what they need without thinking.

13.5 Outside-In Design

Outside-In vs. Inside-Out

Inside-out: Build the system first, then slap a UI on top. Common; usually produces awkward interfaces reflecting internal structure.

Outside-in: Design the interface (the user's experience) *first*, then build the system to support it. The class advocates this approach: it forces the design to serve the user, not the implementation.

13.6 Patterns and Heuristics

Established UI patterns (cards, navigation drawers, modal dialogs, progressive disclosure) are reusable solutions that **leverage user expectations**. The *ui-patterns.com* site and **Nielsen's 10 Usability Heuristics** are common reference points: visibility of system status, match with real world, user control, consistency, error prevention, recognition over recall, flexibility, minimal design, help users recover from errors, help and documentation.

13.7 Practice & Review — UI/UX and Usability

Exam-Style Questions (March 16)

Q1 (MCQ). The primary motivation for designing a system (per lecture) is:

Ans: *Non-functional aspects*. Functionality is taken for granted; non-functional aspects differentiate solutions.

Q2 (Short Answer). Difference between UI and UX?

Ans: UI is the visual/interactive layer (layout, controls, aesthetics). UX is the whole experience — usability, accessibility, performance, feeling. UI is part of UX.

Q3 (Essay). Discuss the security-vs-usability trade-off. How do designers decide where to compromise?

Ans: Designers weigh the *cost of a security breach* against *user friction*. For high-risk contexts (banking, IMS) security wins; for low-risk consumer apps (chat, content), usability

wins. The goal is a conscious decision tied to the threat model, user population, and business impact — not an accident.

Q4 (MCQ). Purpose of personas?

Ans: To understand target users' characteristics and needs so design serves them, not the developer.

Q5 (Short Answer). What is “intuitive” design?

Ans: Design that aligns with users' existing mental models so that interaction requires minimal learning and low cognitive load.

Q6 (Short Answer). Inside-out vs. outside-in design.

Ans: Inside-out builds the system first, UI last (bad). Outside-in designs the user-facing experience first and builds the system around it (recommended).

Q7 (Short Answer). What is cognitive load?

Ans: The mental effort required to use a system. Good design minimises it.

Q8 (Short Answer). What is information architecture?

Ans: The organisation and flow of information across the system from the user's perspective — navigation, grouping, labelling.

14 Behavioral Modeling: Sequence Diagrams & State Machines

Structural vs. Behavioral Models

Class diagrams show **static structure** (what exists). **Behavioral models** show **dynamic behaviour** (how things happen over time): sequence diagrams capture message exchanges between objects; state machines capture how a single entity changes state in response to events.

14.1 Sequence Diagrams

14.1.1 Purpose and Elements

A sequence diagram depicts the **order of interactions** among a set of objects over time.

Element	Meaning
Lifeline	Vertical dashed line representing an object's existence over time. X-axis = objects; Y-axis = time (top → bottom).
Activation Box	Thin rectangle on a lifeline marking the period an object is executing a method.
Message (solid arrow)	A synchronous call — sender waits for a response.
Message (dashed/open arrow)	An asynchronous call / return — sender does not wait.
Fragment	Box enclosing part of the diagram to model conditional logic: alt (if/else), opt (optional), loop , par (parallel).

14.1.2 System Sequence Diagram (SSD) vs. Sequence Diagram

- **System Sequence Diagram:** High-level. Shows interactions between **external actors** and the system as a black box. Used for use-case realisation.
- **Sequence Diagram:** Detailed. Shows internal interactions between **individual objects** inside the system.

14.1.3 Synchronous vs. Asynchronous Messages

- **Synchronous:** Solid filled arrow. Sender blocks until receiver responds. Typical method call.
- **Asynchronous:** Open (stick) arrow. Sender continues immediately. Typical for events, queues, or concurrent processes.

14.2 State Machines

State Machine (Statechart)

A model that describes an entity's behaviour as a set of **states** and **transitions** triggered by **events**. At any moment the entity is in exactly one state.

Element	Meaning
State	A condition or situation (rounded rectangle).
Initial state	Filled dot; where the machine begins.
Final state	Bullseye; where the machine terminates.
Transition	Arrow between states, labelled with event [guard] / action .
Event	The trigger causing a transition.
Guard	Boolean condition that must hold for the transition to fire.
Action	Effect executed during the transition.
Invariant	A property that must remain true while the machine operates.

14.3 Case Study: Submarine Chamber (March 26)

Safety-Critical Modeling: Airlock Chamber

A submarine airlock has an **inner hatch**, an **outer hatch**, and a chamber that can be **flooded** or **drained**. Control panel buttons: **open-inner**, **open-outer**, **close**, **flood**, **drain**, **pause**.

Critical safety invariant: “The inner and outer hatches are **never both open** at the same time.”

States include: *Both closed (dry)*, *Inner open (dry)*, *Outer open (flooded)*, *Flooding*, *Draining*. Transitions between these must **preserve the invariant**; any transition that would break it must be impossible by construction.

14.4 Failure Mode & Effects Analysis (FMEA)

Industry-standard technique: systematically enumerate each component’s possible **failure modes**, analyse the **effect** of each failure, rank by severity $\times$ likelihood, design mitigations (redundancy, alarms, interlocks).

14.5 Formal Verification & Model Checking

Formal Verification

Using **mathematical techniques** to *prove* that a system satisfies safety properties (“nothing bad happens”) and liveness properties (“something good eventually happens”).

Model Checking

Automated exploration of *all reachable states* of a finite model, checking that every state satisfies the specified properties. Representative tools: **Alloy**, **SPIN**, **TLA+**.

14.6 Model-Based Testing

Use the state machine (or other model) to **automatically derive test cases** covering every state and every transition, ensuring systematic coverage rather than ad-hoc test design.

14.7 Practice & Review — Behavioral Modeling

Exam-Style Questions (March 23 & 26)

Q1 (MCQ). Primary purpose of a sequence diagram?

Ans: To model the *dynamic* behaviour of a system — the order of interactions between objects over time.

Q2 (Short Answer). SSD vs. sequence diagram?

Ans: SSD shows the system and external actors (black-box); a sequence diagram shows the internal objects and their messages (white-box).

Q3 (MCQ). Purpose of a lifeline?

Ans: To show the existence and activation of an object across time.

Q4 (Short Answer). Synchronous vs. asynchronous messages?

Ans: Synchronous (solid arrow): sender blocks until reply. Asynchronous (open arrow): sender continues immediately.

Q5 (Essay). Describe state machines and their role in system modeling.

Ans: A state machine models an entity's behaviour as a finite set of states plus transitions triggered by events (optionally guarded, with actions). It is used to specify control-heavy logic, verify safety invariants, and derive tests. The submarine airlock shows how invariants like “both hatches never open” are enforced by construction rather than by runtime checks.

Q6 (Short Answer). What is FMEA?

Ans: Failure Mode & Effects Analysis — systematically enumerate failure modes of each component, assess their effect, and design mitigations.

Q7 (Essay). Formal verification vs. testing. Why does it matter for safety-critical systems?

Ans: Testing samples behaviour; formal verification *proves* properties hold for *all* reachable states. Model checking automates the proof for finite models. In safety-critical systems (airlocks, flight control, medical devices) sampling is insufficient — one unexercised state can kill.

Q8 (Short Answer). What is model-based testing?

Ans: Deriving test cases automatically from a formal model (e.g., a state machine), ensuring coverage of every state and transition.

15 GRASP Patterns & Design Quality

GRASP

General Responsibility Assignment Software Patterns. Principles, not code patterns, that answer the question “**which class should be responsible for what?**” They guide responsibility assignment to achieve low coupling and high cohesion.

15.1 Quality Attributes

Quality Is Subjective

Quality is *multifaceted and subjective* — it varies with stakeholder perspective. Assessing it requires explicit metrics tied to attributes the stakeholders care about. **ISO/IEC 25010** catalogues quality attributes and metrics.

Attribute	Example Metric
Reliability	MTBF, uptime, error rate.
Usability	Task completion time, error recovery, SUS score.
Maintainability	Cyclomatic complexity, coupling/cohesion metrics, time-to-fix.
Performance	Latency, throughput, response time percentiles.
Security	Number of CVEs, time to patch, penetration-test results.
Explainability	Ability of the system to justify its decisions (crucial in AI/credit/medical).

15.2 Accuracy vs. Correctness

Correctness asks: “Is the system producing the *right* output?” **Accuracy** asks: “*How close* is the output to the target?” A system can be highly accurate but still incorrect (wrong spec), or correct but low-accuracy (right model, noisy data).

15.3 The Nine GRASP Patterns

Pattern	Responsibility Rule
1. Information Expert	Assign a responsibility to the class that has the information needed to fulfil it.
2. Creator	Class B should create instances of class A if B <i>contains</i> , <i>aggregates</i> , <i>uses closely</i> , or <i>has initialising data</i> for A.
3. Controller	Assign the responsibility for handling a system event to a non-UI class that represents the overall system or a use-case scenario.
4. Low Coupling	Assign responsibilities so dependencies between classes stay low. Supports reuse and limits ripple effects.
5. High Cohesion	Each class should have a single, focused purpose. Supports understandability and testability.
6. Polymorphism	When behaviour varies by type, assign it using polymorphic operations rather than if/switch on type.
7. Pure Fabrication	Invent a class that does not represent a domain concept when needed to preserve cohesion and lower coupling (e.g., PersistenceService).
8. Indirection	Introduce an intermediate class to decouple two components that would otherwise be directly coupled (e.g., an adapter).
9. Protected Variations	Shield elements from variations in others by wrapping the unstable point behind a stable interface.

15.4 Coupling Metrics: Fan-In and Fan-Out

Metric	Meaning
Fan-In	Number of modules that call a given module. High fan-in $\Rightarrow$ widely reused (usually good, but central; changes are risky).
Fan-Out	Number of modules a given module calls. High fan-out $\Rightarrow$ high coupling, fragile.

15.5 Model-View-Controller (MVC)

Separates concerns into:

- **Model** — data and domain logic.
- **View** — presentation.
- **Controller** — receives input, coordinates the model and view.

MVC exemplifies *Controller*, *Low Coupling*, and *High Cohesion* patterns working together.

15.6 Practice & Review — GRASP & Design Quality

Exam-Style Questions (April 2)

Q1 (Short Answer). Why is assessing software quality difficult?

Ans: Quality is subjective and multi-dimensional — different stakeholders prioritise different attributes (users want usability, ops want reliability, finance wants maintainability). It requires explicit metrics tied to chosen attributes.

Q2 (MCQ). The GRASP **Information Expert** pattern assigns responsibility to:

Ans: The class that already holds the information needed to fulfil it.

Q3 (Short Answer). Explain **fan-in** and **fan-out** and how they relate to quality.

Ans: Fan-in = number of callers; fan-out = number of callees. High fan-out usually indicates high coupling (bad); very high fan-in marks a critical reused module (good for reuse, risky to change).

Q4 (Essay). Discuss design patterns' role in achieving quality, using MVC as the example.

Ans: Patterns encode reusable expert solutions. MVC separates data, presentation, and control, delivering high cohesion within each part and low coupling between them — which directly improves maintainability, scalability, and testability.

Q5 (Short Answer). Accuracy vs. correctness?

Ans: Correctness = producing the right output per spec. Accuracy = how close the output is to a numerical target. High accuracy with a wrong spec = still incorrect.

Q6 (Short Answer). What is explainability and when does it matter?

Ans: The system's ability to justify its decisions. Critical in credit-risk, medical, legal, and AI-driven systems where stakeholders require accountability.

Q7 (Problem-Solving). You are designing a multi-level parking system. Identify three GRASP patterns you would use and why.

Ans: (i) *Creator* — a **Floor** contains **ParkingSpots**, so **Floor** creates them. (ii) *Information Expert* — **Ticket** knows entry time and spot, so **Ticket.calculateFee()** lives there. (iii) *Controller* — a **ParkingController** handles “enter” / “exit” system events, coordinating the domain.

16 Case Studies: When Code Quality Costs Lives

The Central Lesson (April 6)

Not all code is created equally. In safety-critical systems, the difference between “**inferior**” and “**superior**” code is measured not in elegance but in *lives, dollars, and reputation*. As a software engineer you carry real responsibility.

16.1 Critical vs. Safety-Critical Systems

Type	Definition & Example
Critical	Essential for day-to-day operation. Failure causes disruption / financial loss. Example: banking back-end.
Safety-Critical	Failure can directly harm humans or the environment. Example: flight control, radiotherapy, nuclear plant control.

16.2 Therac-25 (1985–1987)

A computer-controlled radiation therapy machine. A software **race condition** (timing bug in the UI and beam-control code) and the removal of hardware interlocks that had existed in predecessors allowed **massive radiation overdoses**. **At least six patients** received lethal or near-lethal doses. **Lessons:** never remove safety interlocks; understand the context when porting code; concurrency bugs need formal analysis.

16.3 Ariane 5, Flight 501 (1996)

The rocket exploded 37 seconds after launch. Cause: a **16-bit to 64-bit** floating-point to integer conversion overflow. The offending inertial-reference code had been *reused* from Ariane 4, whose flight profile never produced the offending value. Loss: **\$370 million**. **Lesson:** reused code must be re-validated in the new operating envelope.

16.4 Boeing 737 MAX MCAS (2018–2019)

The Maneuvering Characteristics Augmentation System (MCAS) pushed the aircraft’s nose down based on a **single angle-of-attack sensor**. A stuck sensor triggered repeated nose-down commands pilots could not override. Two crashes (Lion Air 610, Ethiopian 302) killed **346 people**. Earlier fighter-jet MCAS used **two sensors** with cross-checks. **Lesson:** safety-critical inputs need redundancy; pilots need clear authority and documentation.

16.5 Brake Calibration Incident (Lecturer’s Car)

Brake pressure was tied to engine RPM. Below a threshold RPM, brake assist lost pressure and brakes failed. A **software calibration bug**, not a hardware failure. **Lesson:** cross-domain coupling creates non-obvious failure modes.

16.6 Takeaways for the Software Engineer

Professional Responsibilities

- **Understand existing code before changing it** — most real-world work is brown-field.
- **Test thoroughly**, including edge cases, corner cases, and the operating envelope.
- **Document your reasoning** — even “you” in six months is effectively a different person.
- **Do not blindly trust AI-generated code.** LLM output must be reviewed, understood, and tested.
- **Emergent behaviour is real.** Complex systems can exhibit behaviours no single component produces.
- **Every decision has long-term impact.** Quick fixes become technical debt.

16.7 AI-Assisted Development: Benefits and Risks

LLMs can generate code quickly but introduce risks: **plausible but wrong** output, **lost architectural context**, **subtle security flaws**, **undetected licence issues**. Use AI as an *assistant*, not an *authority*: review every generated block, test it, and understand *why* it works before merging.

16.8 Practice & Review — Case Studies

Exam-Style Questions (April 6)

Q1 (MCQ). Main theme of “Not all code is created equally”?

Ans: The impact of code quality on the **safety** and **reliability** of software systems.

Q2 (Short Answer). Critical vs. safety-critical systems (with an example of each).

Ans: Critical = essential to operation, failure causes disruption (banking). Safety-critical = failure can harm humans (flight control).

Q3 (Short Answer). Lesson of the Therac-25 incident.

Ans: A race condition combined with removing hardware interlocks caused fatal overdoses. Never remove safety redundancy; concurrency bugs need formal analysis; understand context when porting code.

Q4 (Short Answer). Lesson of Ariane 5 Flight 501.

Ans: A 16→64-bit conversion overflow in *reused* Ariane 4 code destroyed the rocket. Reused code must be re-validated against the new operating envelope.

Q5 (MCQ). Key difference between fighter-jet MCAS and 737 MAX MCAS?

Ans: The 737 MAX relied on a single angle-of-attack sensor (no redundancy / cross-check) and mis-handled incorrect data.

Q6 (Essay). Human judgement and oversight with AI-generated code.

Ans: LLMs produce plausible but not always correct code and can lose architectural context. Engineers remain responsible for understanding, reviewing, testing, and integrating. Treat AI as a fast assistant whose output must be verified — never as authoritative especially in safety-critical paths.

Q7 (Short Answer). Deterministic vs. non-deterministic system.

Ans: Deterministic = same input $\Rightarrow$ same output. Non-deterministic = same input can yield different outputs (due to timing, concurrency, randomness).

17 Refactoring

Refactoring

The disciplined process of **restructuring existing code** to improve its internal structure (readability, maintainability, efficiency) *without changing* its external behaviour.

17.1 Lehman's Laws of Software Evolution

Two Laws the Professor Emphasised

1. **Continuing Change:** A system in use must be continually adapted; otherwise it becomes progressively less satisfactory.
2. **Increasing Complexity:** As a system evolves, its complexity *increases* unless explicit work is done to reduce it.

Implication: refactoring is not optional — it is the ongoing work that keeps complexity in check.

17.2 Why Refactor?

- **Reduce technical debt** — the accumulated cost of past shortcuts.
- **Reduce cognitive load** — let future readers (including you) understand the code faster.
- **Enable new features** — clean code is easier to extend.
- **Improve testability** — smaller, cohesive units are easier to test.
- **Reduce defects** — duplication and complexity correlate with bug density.
- **Emerging: reduce energy consumption** — tighter code can lower compute and power use.

17.3 Code Smells Driving Refactoring

Smell	What to do
Long method	<i>Extract Method.</i>
Duplicated code	<i>Extract Method / Extract Class;</i> pull up to a shared location.
Large class	<i>Extract Class;</i> separate responsibilities.
Long parameter list	<i>Introduce Parameter Object.</i>
Divergent change	<i>Extract Class;</i> one class changed for many reasons → split.
Shotgun surgery	<i>Move Method/Move Field;</i> consolidate related changes.
Feature envy	<i>Move Method</i> to the class whose data it uses.
Magic numbers / unclear names	<i>Rename Variable; Replace Magic Number with Symbolic Constant.</i>
Complex conditionals	<i>Decompose Conditional; Replace Conditional with Polymorphism.</i>

17.4 Core Refactoring Techniques

1. **Rename Variable / Method / Class** — names reveal intent.
2. **Extract Method** — group related lines into a well-named helper.
3. **Extract Class** — split responsibilities across classes.
4. **Inline Method / Variable** — remove indirection that adds no value.
5. **Move Method / Field** — relocate to the class that actually uses it.
6. **Introduce Parameter Object** — bundle naturally-grouped arguments.
7. **Decompose Conditional** — turn complex `if` branches into named methods.
8. **Replace Conditional with Polymorphism** — eliminate `switch` on type.
9. **Simplify Conditionals** — merge guards, use early return, remove redundancy.

17.5 Principles of Safe Refactoring

The Golden Rules

1. **Never refactor and extend functionality in the same commit.** Keep behaviour change and structure change separate.
2. **Have a test suite before you start.** Tests are your safety net. If none exist, write characterisation tests for current behaviour *first*.
3. **Take small steps.** Make one atomic change, run tests, commit. Repeat.
4. **Tests remain green after each step.** Red tests $\Rightarrow$ undo or fix before proceeding.
5. **Refactor with a goal.** “Prepare for the next feature” or “eliminate smell X” — not cosmetic tidying.

17.6 Tools

- **Static analysis:** ESLint, SonarQube, Pylint, Checkstyle — flag smells automatically.
- **IDE refactorings:** IntelliJ, VS Code — mechanically safe renames, extractions, moves.
- **Code summarisation / LLM assistants:** useful for understanding legacy code, but *suggestions require verification* (they lose architectural context).

17.7 Design Refactoring

Refactoring is not limited to code. UI and UX also undergo **design refactoring**: the evolution of the Twitter (X) interface across years shows progressive simplification and reorganisation driven by usage data.

17.8 Practice & Review — Refactoring

Exam-Style Questions (April 9 & 13)

Q1 (MCQ). Primary reason for refactoring?

Ans: To improve readability and maintainability *without* changing external behaviour.

Q2 (Short Answer). Two of Lehman’s laws discussed in class?

Ans: *Continuing change* — a used system must keep evolving or decay in usefulness; *increasing complexity* — complexity grows unless explicit work is done to reduce it.

Q3 (Essay). What is a design smell and how does refactoring address it?

Ans: A design smell is a symptom of a deeper design problem (duplicate code, long method, god class). Refactoring applies targeted mechanical transformations (extract method, extract class, move method, etc.) under the cover of a test suite to remove the smell without changing observable behaviour.

Q4 (Short Answer). Why have a good test suite before refactoring?

Ans: Tests are the safety net that detects regressions, letting you restructure aggressively with confidence.

Q5 (Essay). Technical debt and its relation to refactoring.

Ans: Technical debt is the accumulated cost of past shortcuts. Like financial debt, it compounds: every future change pays interest as extra effort. Refactoring is the systematic repayment — small, low-risk principal payments rather than a risky rewrite.

Q6 (MCQ). Main challenge of AI-driven refactoring?

Ans: Loss of architectural context — the LLM does not see the larger design.

Q7 (Short Answer). Purpose of static code analysis in refactoring?

Ans: To identify candidate smells (long methods, duplication, high complexity) automatically and guide where to refactor.

18 Software Maintenance & Legacy Systems

Software Maintenance

The process of modifying a software system **after delivery** to correct faults, adapt to changed environments, improve performance, or prevent future problems.

Maintenance Dominates the Budget

Up to **80%** of total cost of ownership is maintenance, not development. Every design decision is really a maintenance decision.

18.1 The Four Types of Maintenance

Type	Purpose
Corrective	Fix defects found in operation. “The bug.”
Adaptive	Adapt to changes in the <i>environment</i> — new OS, new regulations, new hardware.
Perfective	Improve performance, usability, or structure even though nothing is broken. Includes refactoring.
Preventive	Proactive changes to avoid future problems — tighter tests, documentation, fixing latent code smells.

Memory Aid

C–A–P–P: **C**orrective (broken), **A**daptive (environment), **P**erfective (improve), **P**reventive (future-proof).

18.2 Software Aging

Software, though immaterial, *ages*: accumulated patches, drift between code and documentation, retirement of original developers, and changing user expectations all erode quality over time. Unlike hardware, ageing is addressed by **refactoring and re-engineering**, not by replacement of worn parts.

18.3 Legacy Systems

Legacy System

An outdated but still-operational system that is difficult to modify because of age, obsolete technology, lack of documentation, or loss of the original team.

Real-World Legacy

The US Social Security Administration runs on **~16 million lines of COBOL**. Major banks still run COBOL in core transaction systems. These systems work — but changing them is high-risk and high-cost.

18.4 Modernising Legacy Systems

Option	Description
Scrap & Rewrite	Start over. Highest risk, highest cost, highest potential reward.
Re-engineer	Understand the existing system, restructure it, port to modern tech while preserving behaviour.
Automated Source Code Conversion	Tools translate COBOL → Java etc. Lowest conversion cost; output still needs human review.
Wrap	Put a modern API in front of the legacy system, keep it running behind the scenes.
Do Nothing	Sometimes rational: if it works, is cheap to run, and carries low risk.

18.4.1 The Evaluation Quadrant

Decide among options by rating the system on two axes:

- **Business value** (high / low)
- **System quality** (high / low)

High value + high quality ⇒ keep and maintain. High value + low quality ⇒ re-engineer. Low value + high quality ⇒ minimal maintenance. Low value + low quality ⇒ scrap.

18.5 Reverse Engineering & Forward Engineering

Activity	Direction
Reverse Engineering	Code / binaries → models / design / requirements. Needed to understand undocumented legacy systems.
Forward Engineering	Requirements → design → code. The normal SDLC direction.
Re-engineering	Reverse engineer to understand, then forward engineer a modern replacement preserving behaviour.

18.6 Feathers' 5-Step Process for Changing Legacy Code

From Michael Feathers' *Working Effectively with Legacy Code*:

1. **Identify change points** — where must the code actually change?
2. **Find test points** — where can behaviour be observed and pinned down?
3. **Break dependencies** — introduce seams so the change point can be isolated and tested.
4. **Write tests** — characterisation tests that lock in current behaviour.
5. **Make changes and refactor** — now that tests exist, change with confidence.

18.7 Maintenance Metrics

Metric	Use
Cyclomatic Complexity	Count of linearly independent paths through a method. High values flag hard-to-test, hard-to-maintain code.
Function Points	Functionality-based size measure — independent of language.
Lines of Code (LOC)	Crude but common proxy for size.
Defect Density	Defects per KLOC; quality indicator.
Mean Time To Repair (MTTR)	How fast the team fixes issues.

18.8 Layered Architecture for Maintainability

Organising a system into **layers** (presentation / application / domain / infrastructure), each with a defined interface, allows individual layers to be **modified or replaced independently**. This is one of the simplest and most effective maintainability patterns.

18.9 Practice & Review — Maintenance & Legacy

Exam-Style Questions (April 16)

Q1 (MCQ). What is software aging?

Ans: The need for periodic updates and maintenance to keep software relevant and functional — unlike hardware, the code itself does not physically decay, but the code's relevance, documentation, and fit-for-purpose do.

Q2 (Short Answer). Name the four types of maintenance.

Ans: Corrective (fix defects), Adaptive (environment change), Perfective (improve), Preventive (avoid future problems).

Q3 (Essay). Why is maintenance distinct from earlier SDLC stages?

Ans: Other stages build the system; maintenance keeps a delivered system alive in the face of bugs, changing environments, evolving requirements, and ageing infrastructure. It dominates total cost, runs indefinitely, and forces every change to preserve existing behaviour for current users.

Q4 (MCQ). Goal of *perfective* maintenance?

Ans: To improve performance, functionality, or internal structure when nothing is broken.

Q5 (Essay). Challenges of legacy systems and how re-engineering/modernisation addresses them.

Ans: Legacy systems are hard to change due to complexity, outdated technology, and lost documentation. Re-engineering first reverse-engineers the system to recover understanding, then forward-engineers a modernised version that preserves behaviour, reducing technical debt and opening the way for new features while minimising risk compared to a full rewrite.

Q6 (MCQ). Primary benefit of automated source-code conversion in re-engineering?

Ans: Lower conversion cost than manual rewrite (though the output still needs human review).

Q7 (Short Answer). Reverse engineering vs. forward engineering.

Ans: Reverse engineering goes from existing code/system back to models and requirements. Forward engineering goes from requirements forward through design to code. Re-

engineering combines the two.

Q8 (Short Answer). Cyclomatic complexity — what is it and why use it?

Ans: The number of linearly independent paths through a method. High values indicate code that is hard to test and hard to maintain and therefore a candidate for refactoring.

Q9 (MCQ). Primary benefit of a layered architecture?

Ans: Increased modifiability / scalability — layers can be changed or replaced independently through their defined interfaces.

Q10 (Short Answer). What is the role of documentation in maintenance?

Ans: Documentation captures design intent, architecture, and behaviour, letting future maintainers (who may be strangers to the original code) modify it safely.

19 Exam Traps & Warnings (From Quiz 1 Answer Key)

CRITICAL: Agile is NOT a Development Model!

From the Quiz 1 answer key: “No marks will be awarded for Agile Dev Model. Agile is not a dev model, but rather a **philosophy** (read the agile manifesto). If someone has written Scrum, they get 0. Scrum is a **process** and not a model.”

What the professor wants you to understand:

Term	What it IS
Agile	A philosophy/mindset (the Agile Manifesto has 4 values and 12 principles)
Scrum	A process framework (with sprints, roles, ceremonies)
Waterfall	A development model (sequential phases)
Prototyping	A development model (build prototype → refine)
Evolutionary / Incremental	A development model (build incrementally)
Spiral	A development model / meta-model (risk-driven iterations)

If asked “**which development model?**”, answer with Prototyping, Evolutionary, Incremental, or Spiral. **NEVER** say “Agile” or “Scrum” as a development model!

Quiz 1 Scoring Pattern — What Gets Full Marks

The professor’s scenario questions require you to **name the concept AND explain why**. Partial marks are given:

- Naming the correct concept = **half marks**
- Explaining how/why it applies = **other half**
- **Example:** “The team should use the critical path method (0.5 marks) to identify tasks that directly affect project completion. Understanding slack time (0.5 marks) allows non-critical tasks to be delayed without impacting the timeline.”

The “Magic Keywords” from Quiz 1 Answers

These concept pairs appeared repeatedly in the answer key. The professor expects you to connect them:

Question Theme	Expected Keywords
Development model choice	Prototyping + Evolutionary design
Code quality over time	Refactoring + Open-Closed Principle (or SOLID)
Testing & integration	Continuous Integration + Regression Testing
Scheduling / deadlines	Critical Path Method + Slack Time
Workflow improvements	Process Optimization + DevOps
Design flexibility	Low Coupling + High Cohesion (or SOLID/GRASP)
Maintenance practices	Perfective + Preventive maintenance

20 Crossword Vocabulary — Key Terms & Definitions

Why This Section Exists

Quiz 1 had a **14-mark crossword puzzle** (Question 3) that tested **key vocabulary**. The professor is very likely to include a similar section in the final exam. **Memorize these terms and their definitions!**

20.1 Terms from Quiz 1 Crossword (All Sets)

Term	Definition / Clue
Regression Testing	Re-testing existing functionality after changes to ensure nothing is broken
Critical Path Method	Technique to identify the longest sequence of dependent tasks determining minimum project duration
Continuous Integration	Practice of frequently integrating code changes with automated builds and tests
Slack Time	Time a non-critical task can be delayed without affecting the project end date
Refactor	Restructuring existing code without changing its external behavior
Decomposition	Breaking down a complex system/task into smaller, manageable parts (WBS)
Prototyping Model	Development model that builds a quick mock-up to validate requirements early
Open-Closed Principle	Software entities should be open for extension but closed for modification
Perfective Maintenance	Enhancing existing features and improving performance based on user feedback
Gantt Chart	Bar chart showing project schedule with task durations, start/end dates, and dependencies
Continuous Improvement	Ongoing effort to improve products, services, or processes (PDCA cycle)
Backlog	Ordered list of everything needed in a product (Product Backlog in Scrum)
Alpha Testing	In-house testing by the development team or internal users before external release
DevOps	Combination of Development + Operations for faster, continuous, quality delivery

20.2 Additional Likely Crossword Terms (New Topics Since Quiz 1)

Since Quiz 2 covers **requirements, use cases, conceptual modeling, and testing** (topics taught after Quiz 1), expect these terms:

Term	Definition / Clue
Use Case	Sequence of actions a system performs in response to an actor's interaction
Primary Actor	External entity that initiates an interaction with the system
Precondition	What must be true before a use case can begin
Postcondition	What is true after a use case completes successfully
Include Relationship	Mandatory sub-behavior always used by a use case
Extend Relationship	Optional behavior that adds to a use case under certain conditions
Encapsulation	Hiding internal state; making attributes private with public getters/setters
Multiplicity	Constraint on how many instances can participate in a relationship
Domain Model	High-level representation of key concepts, attributes, and relationships
Equivalence Partitioning	Dividing inputs into equivalent classes and testing one from each
Boundary Value Analysis	Testing at the edges/boundaries of input ranges
State Transition Testing	Testing system behavior as it transitions between states
Unit Testing	Testing individual functions/methods/classes independently
Integration Testing	Testing how combined modules work together at their interfaces
System Testing	Testing the complete integrated system against requirements
Acceptance Testing	End-user testing to validate the system meets business requirements
Stub	Dummy module used in top-down integration to simulate lower modules
Driver	Dummy module used in bottom-up integration to simulate upper modules
MoSCoW	Prioritization: Must-have, Should-have, Could-have, Won't-have
SRS	Software Requirements Specification — the living requirements document
Verification	"Are we building the product right?" — conformance to specs
Validation	"Are we building the right product?" — meeting user needs
MVP	Minimum Viable Product — smallest feature set to satisfy early customers
Traceability	Linking requirements to their origin and forward to design/test
Pesticide Paradox	Running same tests repeatedly won't find new bugs
Corrective Maintenance	Fixing bugs and defects found after release
Adaptive Maintenance	Modifying software to work in a changed environment
Preventive Maintenance	Making changes to prevent future problems
Coupling	Degree of interdependence between modules (want LOW)
Cohesion	Degree to which elements in a module belong together (want HIGH)

21 Scenario-Based Practice Questions (Quiz 1 Style)

How the Professor Designs Questions

Based on Quiz 1, the professor gives a **real-world scenario** and asks you to apply concepts from class. Each sub-question targets a specific topic area. You must **name the concept AND explain how it applies**.

21.1 Practice Scenario 1: Hospital Management System

Scenario: Hospital Management System (5 marks)

A hospital wants to build a new software system to manage patient records, appointments, billing, and pharmacy. Requirements are partially known and will evolve as doctors and staff provide feedback. The system must integrate with existing lab equipment and insurance providers. The team is small (6 developers) and the deadline is 8 months.

- (a) **(1.5 marks)** Which development model should the team adopt? Explain why.

Model Answer: The team should adopt the **Prototyping model combined with Evolutionary/Incremental design**. Since requirements are partially known and will evolve with feedback from doctors, prototyping allows early validation. Incremental delivery allows partial functionality (e.g., patient records first, then billing) to be delivered and tested early.

- (b) **(1.5 marks)** How should design and code quality be managed as new modules (pharmacy, billing) are added?

Model Answer: Code quality should be maintained through **refactoring** (keeping internal structure clean as the system grows) and adherence to the **Open-Closed Principle** (new modules like pharmacy can be added by extending existing interfaces without modifying the patient records module). SOLID/GRASP principles ensure low coupling between modules.

- (c) **(1 mark)** How can testing practices reduce integration risk with external systems (lab equipment, insurance)?

Model Answer: The team should use **Continuous Integration** to frequently merge and test code against the external interfaces, combined with **Regression Testing** to ensure that adding insurance integration doesn't break existing patient record functionality.

- (d) **(1 mark)** How should the team manage the 8-month deadline?

Model Answer: Use the **Critical Path Method** to identify which tasks (e.g., database design → patient module → billing integration) directly affect the deadline. Understanding **slack time** for non-critical tasks (like UI polish) allows flexibility when requirements change mid-project.

21.2 Practice Scenario 2: E-Commerce Platform

Scenario: E-Commerce Platform (5 marks)

A startup wants to build an e-commerce platform for selling handmade crafts. The founding team has a clear vision but the exact feature set is uncertain. They want to launch quickly with basic functionality and iterate based on user feedback. The system must handle payments, product listings, search, and order tracking.

- (a) **(1.5 marks)** Recommend a development approach and justify it.

Model Answer: **Prototyping + Evolutionary model.** The uncertain feature set makes prototyping ideal for validating ideas with real users early. Evolutionary development allows the team to ship an **MVP** (Minimum Viable Product) with core features (listings + payments) and incrementally add search and order tracking based on user feedback.

- (b) **(1.5 marks)** As the platform grows from 100 to 10,000 products, how should design quality be maintained?

Model Answer: Apply **refactoring** to improve code structure as complexity grows, and follow the **Open-Closed Principle** — e.g., the search system should be extensible (add filters by category, price range, rating) without modifying the core search logic. **Low coupling** between modules (payment, search, listings) prevents cascading failures.

- (c) **(1 mark)** How should the team ensure payment integration doesn't break existing features?

Model Answer: **Continuous Integration** with automated builds after each code change, plus **Regression Testing** to verify that payment integration doesn't affect product listing or search functionality.

- (d) **(1 mark)** The team has 12 weeks. How should they plan?

Model Answer: Use **Critical Path Method** to sequence dependencies (database schema → product module → payment → order tracking) and identify the critical path. Tasks with **slack time** (like email notifications) can be deferred if needed.

21.3 Practice: Use Case & Conceptual Modeling Question

Scenario: Room Booking System (6 marks)

A university wants a room booking system where professors can book classrooms, students can book study rooms, and admins can manage room availability.

- (a) **(1 mark)** Identify the primary and supporting actors.

Answer: **Primary:** Professor, Student, Admin. **Supporting:** University Calendar System, Notification Service, Authentication System.

- (b) **(2 marks)** Write a use case for “Book a Room” using the standard template.

Answer:

- **Name:** Book a Room

- **Primary Actor:** Professor / Student
 - **Precondition:** User is logged in; rooms are available
 - **Main Flow:** 1) User selects date/time, 2) System shows available rooms, 3) User selects a room, 4) System confirms booking, 5) System sends confirmation notification
 - **Alternate Flow:** 3a) No rooms available → System suggests alternate times; 4a) Time conflict → System shows error
 - **Post-condition:** Room is booked; appears in calendar
- (c) (2 marks) Identify conceptual classes, their key attributes, and relationships with multiplicity.

Answer:

- **User** (–name, –email, –role) — makes — **Booking** (1..*)
 - **Booking** (–date, –startTime, –endTime, –status) — reserves — **Room** (1)
 - **Room** (–roomNumber, –capacity, –building, –hasProjector) — belongs-to — **Building** (1)
 - **User** 1 — makes — 0..* **Booking**
 - **Room** 1 — has — 0..* **Booking**
- (d) (1 mark) Identify one functional and one non-functional requirement.

Answer: **Functional:** “Users shall be able to search for available rooms by date, time, and capacity.” **Non-functional:** “The system shall respond to search queries within 2 seconds for up to 500 concurrent users.”

21.4 Practice: Testing Question

Testing: Password Validation (4 marks)

A system accepts passwords that must be 8–20 characters long, contain at least one uppercase letter, one digit, and one special character.

- (a) (1 mark) Apply **Equivalence Partitioning** for the length constraint.

Answer:

- **Valid:** 8–20 characters (e.g., “Pass@1234” — 9 chars)
- **Invalid 1:** < 8 characters (e.g., “Pa@1” — 4 chars)
- **Invalid 2:** > 20 characters (e.g., 25-char string)

- (b) (1 mark) Apply **Boundary Value Analysis** for the length constraint.

Answer: Test with: 7 chars (invalid), 8 chars (valid boundary), 9 chars (valid), 19 chars (valid), 20 chars (valid boundary), 21 chars (invalid)

- (c) (1 mark) Design a **State Transition** test for login with this password system (3 attempts allowed).

Answer: Idle $\xrightarrow{\text{enter password}}$ Checking $\xrightarrow{\text{correct}}$ Authenticated. Checking $\xrightarrow{\text{wrong}}$ Attempt 2 $\xrightarrow{\text{wrong}}$ Attempt 3 $\xrightarrow{\text{wrong}}$ Account Locked.

- (d) **(1 mark)** What integration testing strategy would you use if the password module depends on the database module and the authentication module?

Answer: **Bottom-up integration:** Test the database module first (lowest level), then integrate with the password module (use a **driver** to simulate the authentication module), then integrate all three.

22 Additional Practice Questions

22.1 Short Answer Questions

Q1: Definitions — Answers

1. **What is Software Engineering?**

The application of systematic, disciplined, and quantifiable approaches to the **development, operation, and maintenance** of software. It applies classical engineering principles (planning, rigor, measurement) to software in order to produce reliable, efficient, and maintainable products on time and within budget.

2. **What is the difference between Green Field and Brown Field systems?**

Green Field: Building a new system *from scratch* — no legacy constraints, full freedom in technology and architecture choices.

Brown Field: Building on or integrating with *existing* systems. Developers must work around legacy constraints, technical debt, existing data formats, and deployment environments. Most real-world work is brown field.

3. **Define the SDLC and name its phases.**

The **Software Development Life Cycle (SDLC)** is the structured end-to-end process for planning, creating, testing, deploying, and maintaining software.

Phases: **(1)** Feasibility Study → **(2)** Requirements Analysis → **(3)** System Design → **(4)** Implementation → **(5)** Testing → **(6)** Deployment → **(7)** Maintenance.

4. **What is the PDCA cycle?**

Plan–Do–Check–Act: an iterative quality improvement loop.

- **Plan:** Identify the goal and design the change.
- **Do:** Implement on a small scale.
- **Check:** Measure results against expectations.
- **Act:** Standardise the change if successful; restart if not.

5. **What is the difference between Verification and Validation?**

Verification — “Are we building the product *right*?”: checking that the software conforms to its specification. Done internally via reviews, inspections, unit tests.

Validation — “Are we building the *right* product?”: checking that the software actually meets the user’s real needs. Done at delivery via acceptance testing (UAT).

Memory aid: Verification = spec compliance; Validation = customer satisfaction.

6. **Define: Use Case, Actor, Conceptual Model.**

Use Case: A description of a sequence of interactions between the system and an actor that achieves a goal of value to that actor.

Actor: A role played by a person, organisation, or external system that interacts with the system (but is outside it).

Conceptual Model (Domain Model): A diagram of key domain concepts (classes), their attributes, and associations — purely representational, implementation-independent.

7. **What is the Agile Manifesto? Name its four values.**

A declaration (2001) of 17 signatories establishing values for lightweight, iterative software development.

Four values (prefer the left side, but value the right):

1. **Individuals and interactions** over processes and tools
 2. **Working software** over comprehensive documentation
 3. **Customer collaboration** over contract negotiation
 4. **Responding to change** over following a plan
8. **What is a Function Point?**
A **language-agnostic unit of software size** measured from the user's perspective. Counted by weighting five component types (external inputs, external outputs, external queries, internal logical files, external interface files) by complexity. Used to estimate development effort, cost, and productivity independently of implementation language.
9. **What is the Critical Path in a project schedule?**
The **longest chain of dependent tasks** from project start to end. Tasks on the critical path have **zero float (slack)** — any delay in one critical task delays the entire project completion date by the same amount.
10. **What is the difference between Black Box and White Box testing?**
Black Box (Functional): Tests the system from the *outside* — only inputs and expected outputs matter; no knowledge of internal code. Techniques: Equivalence Partitioning, Boundary Value Analysis.
White Box (Structural): Tests *internal* code paths, branches, and logic flows. Requires source code access. Techniques: Statement coverage, Branch coverage, Path testing.
11. **What is the Open-Closed Principle? Give an example.**
OCP (SOLID): Software entities should be **open for extension** but **closed for modification**. New behaviour is added by writing new code, not by changing existing code.
Example: An 'AreaCalculator' calls a polymorphic `area()` method on a 'Shape' interface. Adding a 'Triangle' shape requires only a new class implementing 'Shape.area()' — the calculator is never touched.
12. **What are the four types of software maintenance?**
 1. **Corrective:** Fixing defects and bugs reported post-release.
 2. **Adaptive:** Adapting the system to a new environment (new OS, hardware, regulations, APIs).
 3. **Perfective:** Improving functionality, performance, or usability beyond the original spec.
 4. **Preventive:** Restructuring/refactoring to reduce technical debt and prevent future failures.

Mnemonic: CAPP.
13. **What is the difference between a Stub and a Driver?**
Stub: A simplified *replacement for a called module* (below the module under test). Used in **top-down** integration — simulates lower-level components not yet built.
Driver: A component that *calls* the module under test (acts as the upper-level caller). Used in **bottom-up** integration — simulates the higher-level code not yet built.
Rule: Top-down needs stubs; bottom-up needs drivers.

14. What is the Pesticide Paradox in testing?

Beizer's 5th principle: if the **same test cases are run repeatedly**, they eventually stop finding new defects — just as insects become resistant to the same pesticide. To keep finding bugs, testers must **regularly review, revise, and write new test cases**. The implication is that a static test suite offers a false sense of security over time.

22.2 Comparison Questions

Q2: Compare and Contrast — Answers

1. Waterfall vs. Agile

Aspect	Waterfall	Agile
Structure	Sequential rigid phases	Iterative sprints
Requirements	Fixed at start	Evolve throughout
Customer involvement	Start & end only	Continuous
Documentation	Comprehensive, upfront	Minimal, just-enough
Risk exposure	High (errors found late)	Low (feedback each sprint)
Best for	Stable, well-defined specs	Changing requirements

2. Functional vs. Non-functional requirements

Functional (FR): Describe *what* the system does — specific behaviours, features, and functions (e.g., “the user shall be able to log in with email and password”). Testable via specific feature tests.

Non-functional (NFR): Describe *how well* the system performs — quality attributes such as performance, security, scalability, usability, reliability (e.g., “the login response shall be <1s at 10 000 concurrent users”). Testable via load tests, security audits, accessibility checks.

Key distinction: removing a functional requirement removes a feature; violating an NFR makes the whole system unacceptable even if features work.

3. Unit, Integration, System, and Acceptance Testing

Level	Scope	Done By	Goal
Unit	Single function/class	Developer	Verify component logic
Integration	Module interactions	Dev/Tester	Verify interfaces and data flow
System	Entire system	Tester	End-to-end behavior vs. spec
Acceptance	Full system	Customer/User	Meets business needs (UAT)

4. Primary Actors vs. Supporting Actors

Primary Actor: *Initiates* the use case to fulfil a personal goal. The use case exists to serve this actor. Drawn on the **left** of a use case diagram (e.g., Customer placing an order).

Supporting Actor: Provides a service *to* the system at the system's request; the system initiates contact. Drawn on the **right** (e.g., Payment Gateway, Email Server). Key distinction: primary actors drive the interaction; supporting actors respond to

it.

5. <<include>> vs. <<extend>> in Use Cases

«**include**»: **Mandatory** — the base use case *always* invokes the included use case. Models a shared, reusable sub-flow (e.g., every “Checkout” always “Validates Payment”).

«**extend**»: **Optional/conditional** — the extending use case adds behaviour to the base only under specific conditions. Models exceptional or optional behaviour (e.g., “Apply Discount” extends “Checkout” only if a coupon code is entered).

Arrow direction: include points *from* base *to* included; extend points *from* extension *to* base.

6. Big Bang vs. Top-Down vs. Bottom-Up Integration Testing

Big Bang: Integrate all modules simultaneously, then test. Simple, no stubs/drivers needed. Hard to isolate failures; not recommended for large systems.

Top-Down: Test from the top module downward. Requires **stubs** for lower modules. Architecture validated early; I/O modules tested last.

Bottom-Up: Test from low-level modules upward. Requires **drivers** for upper modules. Infrastructure tested first; user-facing integration comes last.

Hybrid (Sandwich): combines both directions to reduce stub/driver count.

7. Equivalence Partitioning (EP) vs. Boundary Value Analysis (BVA)

EP: Divides the input domain into *equivalence classes* where all values in a class should behave identically. Tests one representative value per class (valid & invalid). Reduces redundant test cases.

BVA: Tests values at the *edges* of each partition (minimum, maximum, min−1, max+1), since boundaries are historically where most off-by-one errors occur.

Relationship: complementary — EP identifies the partitions; BVA probes their boundaries. Use both together.

8. Conceptual Classes vs. Software Classes

Conceptual (Domain) Class: Represents a real-world concept in the problem domain (e.g., “Order”, “Customer”). Has attributes meaningful to the domain. *No methods*. Created during Analysis. Lives in the Conceptual Model.

Software Class: An implementation entity. Has methods, visibility modifiers, constructors, and may reference design patterns. Created during Design/Implementation. Maps directly to source code.

A conceptual class *inspires* a software class but is not identical to it.

9. Corrective vs. Perfective vs. Preventive Maintenance

Corrective: *Reactive* — fixes defects reported after release. Restores intended behaviour. Triggered by a bug report. (e.g., patching a crash on null input.)

Perfective: *Proactive improvement* — enhances features, performance, or usability beyond the original spec. Not triggered by a bug. (e.g., optimising a slow database query.)

Preventive: *Proactive risk reduction* — restructures fragile or complex code to prevent future failures before they occur. (e.g., refactoring a 2,000-line god class into smaller, cohesive modules before adding new features.)

10. Coupling vs. Cohesion

Coupling: Degree of *inter-module* dependency. Desired: **LOW**. High coupling means changing module A forces changes in B, C, and D — fragile, hard to test,

hard to reuse.

Cohesion: Degree of *intra-module* unity — how closely related and focused the elements within a single module are. Desired: **HIGH**. High cohesion means each module has one clear responsibility, making it understandable and maintainable.

Golden rule: *Maximise cohesion; minimise coupling*. These two goals reinforce each other — a module with a single responsibility naturally has few reasons to depend on unrelated modules.

22.3 Multiple Choice Quick Review

Q3: MCQ Practice

1. The first step in SDLC is:
A) Coding B) Design C) **Feasibility Study** D) Testing
2. The #1 reason for software project failure is:
A) Bad coding B) **Poor requirements** C) No testing D) Bad UI
3. Which is NOT an Agile value?
A) Working software B) Customer collaboration C) **Comprehensive documentation** D) Responding to change
4. Maintenance typically takes what percentage of total effort?
A) 20% B) 40% C) 60% D) **70–80%**
5. Which testing technique tests at the boundaries of input ranges?
A) Equivalence Partitioning B) **Boundary Value Analysis** C) State Transition D) Path Coverage
6. A primary actor in a use case:
A) Provides services B) **Initiates the interaction** C) Is always a human D) Is optional
7. Refactoring means:
A) Adding new features B) Fixing bugs C) **Restructuring code without changing behavior** D) Writing tests
8. The Spiral model is best suited for:
A) Small projects B) **High-risk projects** C) Simple requirements D) Solo developers
9. In Earned Value Management, $CPI = EV/AC$. If $CPI > 1$:
A) Over budget B) **Under budget** C) On schedule D) Behind schedule
10. Which multiplicity means “one or more”?
A) 0..1 B) * C) **1..*** D) 0..*
11. Agile is best described as:
A) A development model B) A process framework C) **A philosophy/mindset** D) A testing technique
12. The Open-Closed Principle states that software should be:
A) Open for modification, closed for extension B) **Open for extension, closed**

- for modification** C) Open for testing, closed for deployment D) Open source and closed source
13. Perfective maintenance is about:
A) Fixing bugs **B) Enhancing features and performance** C) Adapting to new OS D) Preventing future errors
14. In top-down integration testing, what replaces lower-level modules?
A) Drivers **B) Stubs** C) Mock objects D) Test harnesses
15. A task on the critical path has slack time of:
A) Undetermined B) Maximum **C) Zero** D) Depends on project size
16. Which of these is a CORRECT development model for uncertain requirements?
A) Waterfall **B) Prototyping** C) Agile D) Scrum

23 Quick Reference Cheat Sheet

SDLC Phases (In Order)

Feasibility → **R**equirements → **D**esign → **I**mplementation → **T**esting → **D**eployment → **M**aintenance

*Remember: **F**riendly **R**obots **D**on't **I**gnore **T**esting **D**uring **M**aintenance*

Agile Manifesto (4 Values) — AGILE IS A PHILOSOPHY!

1. **Individuals & interactions** > processes & tools
2. **Working software** > comprehensive documentation
3. **Customer collaboration** > contract negotiation
4. **Responding to change** > following a plan

Testing Levels

Unit → **I**ntegration → **S**ystem → **A**cceptance

*Remember: **U** Investigate **S**oftware **A**lways*

Good Requirements = CCUCFVT

Correct, **C**omplete, **U**nambiguous, **C**onsistent, **F**easible, **V**erifiable, **T**raceable

Design Principles Cheat Sheet

- **SOLID** = Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
- **OCP** = Open for Extension, Closed for Modification
- **Goal**: Low Coupling + High Cohesion
- **Maintenance Types (CAPP)**: **C**orrective (bugs), **A**daptive (environment), **P**erfective (enhance), **P**reventive (future-proof)

GRASP — 9 Patterns

Information Expert · **C**reator · **C**ontroller · **L**ow Coupling · **H**igh Cohesion · **P**olymorphism · **P**ure Fabrication · **I**ndirection · **P**rotected Variations

UI vs. UX & Design Trade-offs

UI = visual/interactive layer. **UX** = whole experience (UI is a subset). Design is the art of making conscious trade-offs: Security ↔ Usability, Performance ↔ Maintainability, Privacy ↔ Functionality. Prefer **outside-in** design.

Behavioral Modeling Quick Reference

- **Sequence diagram:** lifelines + activation boxes + messages (solid = sync, open = async); fragments for `alt/opt/loop/par`.
- **State machine:** states + transitions labelled `event [guard] / action`; initial/final states; invariants.
- **SSD** $\neq$ sequence diagram: SSD is black-box (actor $\leftrightarrow$ system); sequence diagram shows internal object interactions.

Refactoring Golden Rules

1. Never refactor + add features in the same commit.
2. Have a test suite first (write characterisation tests if none exist).
3. Small steps; tests green after each.
4. Refactor with a goal (prepare for feature X or remove smell Y).
5. Lehman: *continuing change & increasing complexity* make refactoring inevitable.

Famous Failure Case Studies

Incident	Lesson
Therac-25	Never remove safety interlocks; concurrency bugs are fatal.
Ariane 5 (Flight 501)	16→64-bit overflow in reused code; re-validate in new envelope.
Boeing 737 MAX (MCAS)	Safety-critical inputs need redundancy + cross-checks.
Knight Capital (2012)	Follow deployment process; one bad update = business loss.

Key Numbers to Remember

Fact	Number
Lines of code moved to production per day (avg)	25 lines
Maintenance effort	70–80% of total
Development effort	20–30% of total
Project success rate	Only 10–20%
Mahindra XEV 9e lines of code	75 million
PDCA steps	Plan, Do, Check, Act
MoSCoW categories	Must, Should, Could, Won't
VAF range	0.65 to 1.35

V&V Quick Reminder

- **Verification:** “Are we building the product **right**?” (process-oriented)
- **Validation:** “Are we building the **right** product?” (product-oriented)

Quiz 1 Keyword Pairs — MEMORIZE THESE

If they ask about...	Answer with...
Development model	Prototyping + Evolutionary
Code quality	Refactoring + Open-Closed Principle
Testing practices	CI + Regression Testing
Scheduling	Critical Path + Slack Time
Workflow speed	Process Optimization + DevOps

Good Luck on Your Exam!

Remember: Understanding concepts is more important than memorizing.
Think about **WHY** things are done, not just **WHAT** is done.

“Software engineering is about **people first**, not technology first.”

Don't say Agile is a development model. Don't say Scrum is a model.
Always pair concept name + explanation for full marks!